

CPSC 317: Introduction to Computer Networking

Sept. 3rd - Module 0: Introduction and Internet Design

- Instructor: Norm Hutchinson & Ibtissem
 - norm@cs.ubc.ca
 - Ibtissem.bouacheria@ubc.ca
 - cpSC317-admin@cs.ubc.ca
- Tutorials (Optional - Credit free)
 - Practical exercises based on course content and programming assignments
 - 10 tutorials, and starting this week
- iClicker (5% Bonus Grade - half for answering, half for answering correct)
- PrairieLearn
 - Assignments: 35%, 5 PAs (all individually except PA4)
 - Quizzes: 30%, 5 Quizzes (Biweekly, starting from Sep. 22nd)
 - Final Exam: 35%
- Must get 50% in the weighted average of the quizzes and final exam; must get 50% on the assignments
- Reflection
 - How do we make - communication media, network architecture, protocols, applications - work?
 - What is a computer network used for?
 - Communication, information sharing, task distribution, resource sharing, scaling and redundancy
- Coping with complexity
 - Complexity limits the size of the systems we build (need techniques to cope with it in order to build larger systems)

- Modularity: divide system into smaller components
- Abstraction: define how a component interacts with the outside world, and don't worry about how it works internally
- Robustness and independence
 - be tolerant of inputs
 - be strict on outputs
- Layering: type of modularity, system contains a number of levels (each built on the previous one), used in networks and OS
- Hierarchy: more general approach when layering is too strict
- Indirection: names - hold systems together, allows sub-systems to be swapped more easily
- Why computer networking?
 - Focus on **communication** - requires at least 2 parties
 - Plumbing
- Objectives
 - writing and working with different types of programs that use computer networks
 - terminology about networking
 - Key paradigms: isolation and privacy, data loss, performance, naming and location, using layers and abstractions
 - application of strategies and paradigms
 - concepts about Internet
- Protocol Stack
 - Application - HTTP (web), email, file transfer, multimedia
 - (OS) Transport - TCP, UDP, from one **application** to another **application**
 - (OS) Network - IP, from one **device** to another **device**
 - (Hardware) Link - Ethernet, formatting of information
 - (Hardware) Physical - 802.11b/g/n, 1000BASE-T, how to encode/send a bit (Electrical Engineering part)

Sept. 5th - Module 1.1: Design of the Internet

- What is the Internet?
 - In the beginning, consider a variety of private networks
 - Internet: network of networks
- The Internet: "nuts and bolts" view
 - **Hosts, end-systems**: billions of connected computing devices
 - **Communication links**: fiber, copper, radio, satellite; varying *bandwidth/latency*
 - **Routers**: forward *packets* (chunks of data)
 - **Regional, local, company networks**
- Internet Goals:
 - Main goal: integrating a number of separately administered entities into a common entity
 - Other goals: network-loss-safe, multiple-type-devices, multiple-type-networks, distributed management, cost effective, easy host attachment, accountable resources
- Communication
 - Necessary condition: medium, same "language" (protocol), 2+ entities (source and destination), message
 - Large scale "communications" system: telephone / telegraph, mail / package delivery
- Analogy: Mail delivery
 - postal address: name, street number, street name, etc
- Analogy: Phone system
 - phone number $\approx$ address
- How does the analogies translate to the Internet
 - Internet address: traditionally, one IPv4 address $\rightarrow$ the Internet $\rightarrow$ another IPv4 address
- How are IP addresses organized?
 - Best answer: by organization
- Consider n devices
 - To connect one to every other device: need $O(n^2)$ wires
 - To reduce wires, consider a **switched network**: different devices connect

to a connection medium (shared)

- Multiplexing
 - Data flows need to be "multiplexed"
 - multiple input streams must share the medium
 - it must be possible to "demultiplex" at the destination
 - Circuit switching: FDM and TDM
 - FDM - Frequency division multiplexing: optical, electromagnetic frequencies divided into (narrow) frequency bands; each call allocated to its own band, can transmit at max rate of that narrow band
 - TDM - Time division multiplexing: time divided into slots
 - Other methods: Code DM, Orthogonal multiplexing (combination of techniques)
- How is data sent?
 - Option 1: Circuit switching
 - dedicated path between source and destination
 - the path the data will take is determined when the connection is established
 - single stream of information per path
 - data flows need to be "multiplexed": multiple input streams must share the medium
 - Pros: guaranteed performance once the circuit was established, immediate data transmission once the call began, no delay
Cons: wasteful for burst-y traffic, connection/setup time is overhead, poor fault tolerance
 - Option 2: Packet switching (much less "obvious" option)
 - data is divided in packets (small chunks of data) that are sent individually
 - Self contained Internet packet does not need to include the route to destination
 - each packet contains a **header** and **payload**
 - independently routed from source to destination
 - Pros: burst-y traffic, statistically good performance is good enough, starting a new conversation is frequent

Cons: queues → loss of packets

- Summary
 - Internet: network of networks
 - Design goals
 - Circuit vs. Packet switch

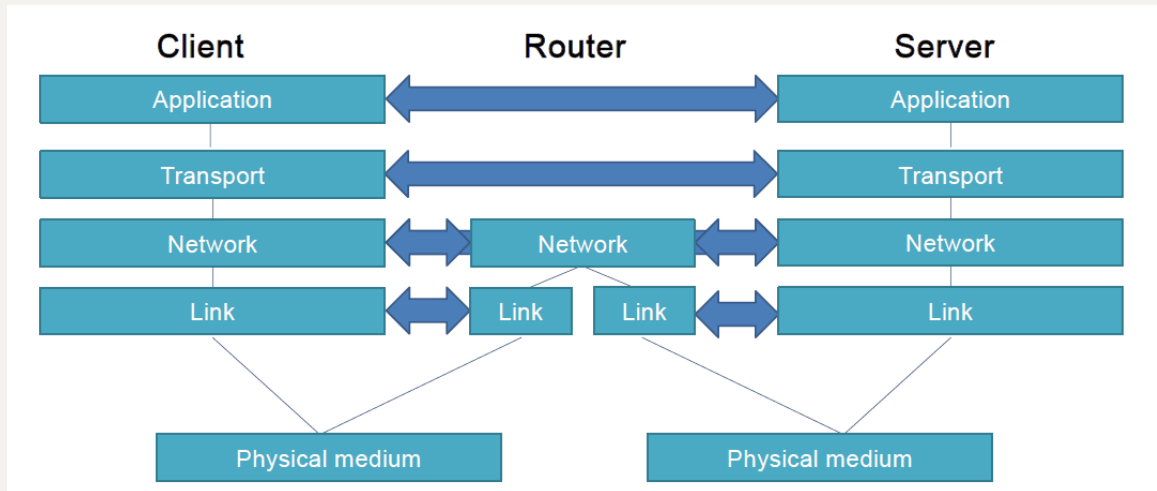
Sep. 8th - Module 1.2: Switching and Protocols

- Packet switching and routers
 - packet is sent to the router that is believed to be closest to the destination
 - router looks up destination address in a forwarding table to determine next hop
 - there may be several possible paths to take
- Circuit vs. packet switching revisited
 - circuit switching, the decision about the route that the data will take from source to destinations is made once when the connection is established
 - packet switching, this decision is made for every packet
 - Reliability?
 - Performance?
- Clicker question - assume A creates a 100 kbps circuit to B, and A sends data at an average rate of 25 kbps, what is A's utilization of the network resources?
 - 25%
- Clicker question - can the network above use these idle resources for other traffic?
 - No
- What is a protocol?
 - A protocol defines:

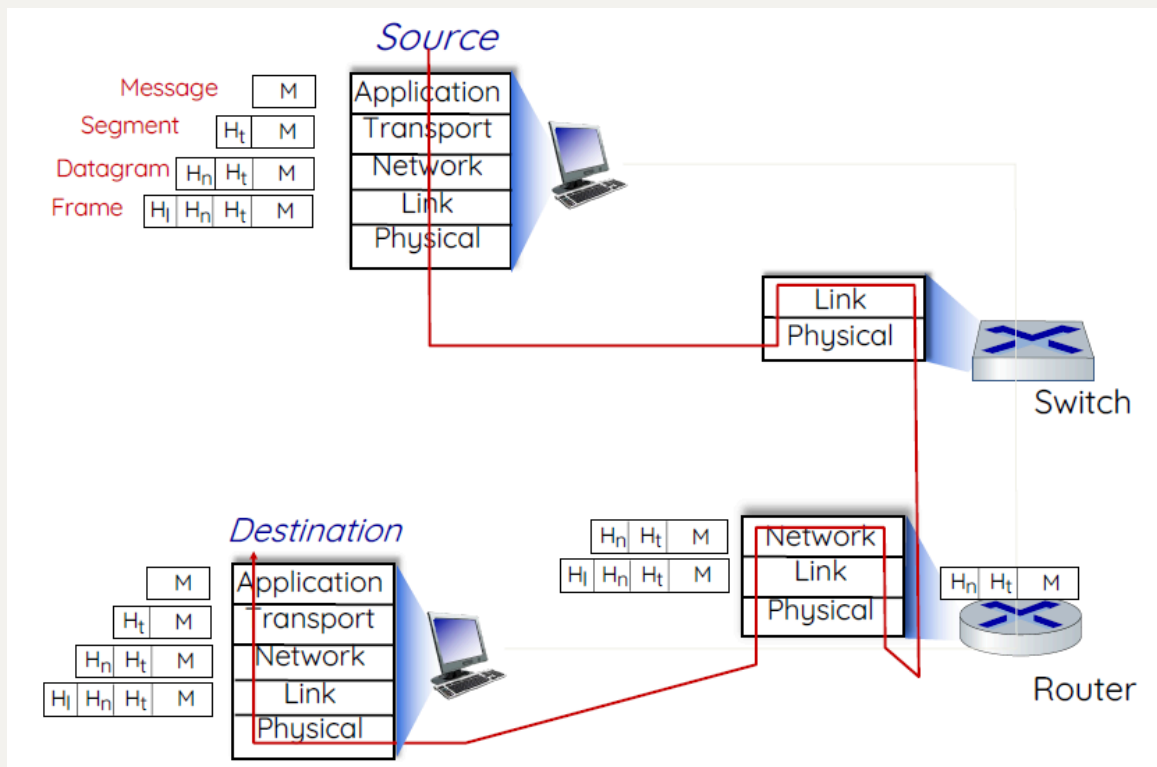
Roles of communicating entities, format of messages, order of messages, actions taken on the transmission, receipt of a message, or other event
 - A fully-defined protocol must provide a proper action for **any event** in **any state**
- Request-response protocol
 - Requestor sends a request

- Receiver sends a response
- Well-defined rules for whose turn it is
- Server can be slow to respond, size of request or response can vary
- Modelling protocols: (linked) finite state machines
- Protocol Stack
 - Application - Transport - Network - Link - Physical
 - Letter analogy
 - application layer protocol: writing a letter using the proper format expected by the recipient
HTTP (web), Email, file transfer, multimedia
 - transport layer protocol: handling reliability and making sure the message reaches the correct person
name $\approx$ port number
TCP, UDP
 - network layer protocol: like a routing officer at a post office or logistics center who decides the best sequence of roads, cities, or hubs to move the letter toward its goal
devising a plan of "delivery"
IP
 - link layer protocol: each local delivery
individual one step in the "delivery process"
Ethernet
 - physical layer protocol: make sure the letter is in the right container suitable for transport medium
actual delivery medium
[some frequency]
- Sending data through the stack
 - Consider client and server
 - application layer thinks they talk with each other
transport layer thinks talk with each other
 - network layer cannot just talk with each other, needs to talk with routers in between
link layer also need to go through the link layers of routers

- link layers use the physical medium to send it



- Data bounces between network, link and physical multiple times
- Encapsulation
 - matryoshka dolls: message, segment, datagram, frame
- Protocol layering and data
 - each layer takes data from above
 - adds header information to create new data unit
 - passes new data to layer below



- Routers can be altering headers
- Protocol stack: responsibilities
 - transport: identifies process on machine; ensures data arrives in order;

recovers lost data

- network: routes datagrams through routers to destination machine
- link: routes frames to adjacent machines ("direct" connection)
- physical: encodes data appropriately for the physical medium

Sep. 10th - Module 2.1: Network Performance

- Network metrics
 - bandwidth, throughput, goodput, latency, round-trip-time, jitter
- Bandwidth
 - maximum rate at which data can be sent over a link
 - e.g. 1 Gbps ethernet, 500 Mbps or 1 Gbps "speed" from Shaw, 2500 Mbps "speed" from Telus
 - advertised rate is almost always the bandwidth
- Units of measurement
 - Data size: [K/M/G] [b/B]
 - $K = 2^{10}$, $M = 2^{20}$, $G = 2^{30}$
 - Data rates: [K/M/G] bps
 - $K = 10^3$, $M = 10^6$, $G = 10^9$
 - 1 B = 8 b
 - Discrepancy grows with size
- Throughput
 - amount of data moved from one location to another in a given time
 - usually expressed in bps
- Goodput
 - rate at which *useful* data arrives
 - does not include headers and encoding costs
 - does not include data loss and retransmissions
 - may depend on context and application-layer protocol
- Latency
 - delay from when something is sent until it is received
 - "something" depends on the context, but must be consistent

- packet latency: from start of sending packet until completely received
- bit/byte latency: from start of sending bit/byte until that bit/byte is completely received
- Round-trip Time (RTT)
 - latency for sending something and receiving something back
 - latency for message + latency for response + processing time
 - RTT for a bit $\neq$ RTT for a request and response
 - easier to compute than one-way latency
 - reported by `ping`, `tracert`
- Jitter
 - variation in latency and/or RTT
 - different paths for packets, network congestion, not implementing packet prioritization, poor hardware performance (old equipment), wireless jitter (interference in medium)

Sep. 12th - Module 2.2: Delay

- Bottlenecks
 - Traffic flow cannot flow at maximum bandwidth in all links
- Type of delay
 - Processing: examine packet to decide where to direct it
 - mainly fixed
 - same sized packets to the same destination: fixed
 - Queueing: waiting time to get access to the link
 - variable
 - same sized packets to the same destination: variable
 - Transmission: time to actually write the packet onto the medium
 - fixed (1 bit) and variable (different-sized bits)
 - same sized packets to the same destination: fixed
 - Propagation: time spent to move each bit from source to destination on the transmission medium
 - depends on the distance
 - same sized packets to the same destination: fixed

- End-to-end: sum of all sources of delay
- Propagation
 - Satellites: 750 km away from surface of Earth, $c = 3 \times 10^8$ m/s, packets = 1250 bytes
 - Two-way propagation delay: $\frac{750 \times 1000}{3 \times 10^8} = 250 \times 10^{-5}$ s, the two way is 5 ms
- Transmission
 - transfer rate is 100 Mbps
 - one-way transmission delay is $\frac{1250 \times 8}{100 \times 10^6} = 0.1$ ms
- End-to-end
 - ACK: server's response - an "acknowledgement"
- Traffic intensity
 - Determined by
 - number of packets arriving per second (a)
 - average packet size (L) in bits
 - transmission rate: rate at which bits are disposed off per second (R)
 - $\frac{L_a}{R}$
 - $L_a = R$ means the network is balanced
 - $L_a > R$ means having packet loss
 - $L_a < R$ means the router can handle the data flow
 - $L_a \ll R$ means the router are rather idle
- Traffic Intensity vs. Queueing Delay
 - assuming packets arrive at an exponential distribution, delay is given
 - $\frac{S}{1-U}$
 - where S is average service time when server is idle, U is server utilization (usually traffic intensity)
 - Queueing delay is then
 - $\frac{S}{1-U} - S$ or $\frac{US}{1-U}$
 - Routers do not have infinite buffer space
 - Packets also can be corrupted in transit

Sep. 15th - Module 3.1: Application Layer Protocols

- Design consideration for application-layer protocols
 - each application using the network will define its own protocol
 - open vs. proprietary
 - architecture: client-server, P2P (peer-to-peer)
 - choice of transport protocol
 - types and formats of messages
- Open vs Proprietary protocols
 - open: publicly known
 - example: DICT, HTTP, SMTP, SSH
 - usually defined in RFC (request for comments) documents
 - many different implementations
 - proprietary
 - example: Skype, iCloud, Zoom
 - only one implementation
- Client-server architecture
 - well-defined roles for client and server
 - server is always on, with permanent address or host name
 - client establishes connection
 - connection is always between one client and one server (server will serve multiple clients at once)
- Peer-to-peer architecture
 - connections typically between peers with the same hierarchical role
 - some hierarchy may be used, but connection is not restricted to it
 - peers request service from other peers, provide service in return
 - self scalability: new peers bring new demand and new capacity
 - complex peer address management
- What quality of service does an application need?
 - Data loss
 - some apps can tolerate some loss (e.g. audio)

- other apps require 100% reliable data transfer (e.g. file transfer, web, email)
- Time sensitivity
 - some apps require low delay to be "effective" (e.g. interactive ones)
- Bandwidth
 - some apps (e.g. multimedia) require minimum amount of bandwidth to be "effective"
 - other apps ("elastic apps") make use of whatever bandwidth they get
- Two transport options
 - Simple, fast; but unreliable, no order guarantee
 - Reliable, in-order delivery, congestion control; but slow
 - TCP - Reliable stream
 - connection, reliable ordered delivery, flow/congestion control, possible delays
 - UDP - Unreliable packet
 - no connection, best effort, no flow control, no (transport level) delay
- Application
 - file transfer, web, email (TCP)
 - loss averse, not time sensitive, elastic bandwidth
 - text messaging (TCP or UDP)
 - loss averse, elastic bandwidth, somewhat time sensitive
 - on demand multimedia streaming (TCP or UDP)
 - some loss tolerance, somewhat time sensitive
 - real time multimedia, VoIP, interactive games (UDP)
 - some loss tolerance, time sensitive, bandwidth requirements
 - domain name service (UDP)
 - loss tolerant, not time sensitive, elastic bandwidth
- The DICT protocol
 - Defined in RFC 2229 (Google: dict protocol rfc)
 - Simple text-based, request-response protocol

- commands: help-define, match, show db, show strat, quit
- e.g. `netcat dict.org 2628` (2628 is port number)
- Transport layer address
 - a pair of a 32-bit IP host address and a 16-bit port number
 - Usually the IP address is derived from a DNS name
 - dict.org → DNS name; 2628 → port number
- Socket - network endpoints
 - created via `socket()` system call
 - destroyed via `close()`
 - data sent and received via `send()` and `recv()`

Sep. 17th - Module 3.2: The Web

- The World Wide Web
 - one of the first Internet applications that became popular
 - information available on the web in the form of a "web page"
 - a web page consists of objects
 - a base HTML file refers to other referenced objects
 - images, scripts, stylesheets, frames
 - each object (including base file) is addressable by a URL
- HyperText Transfer Protocol (HTTP)
 - HTTP is the Web's main application layer protocol
 - Client/Server model
 - uses TCP, typically port 80 (443 for HTTPS)
 - For each Web object
 - client sends one request message at once
 - server sends full response message at once
 - Stateless: server maintains no information about past requests
- HTTP message format
 - Request is formatted using text in ASCII
 - first line: method, URL, version
 - following lines use format "header-name: value"

- lines end with CR-LF
 - empty line ends request header
 - if body needed, it follows the header (size determined by "content-length" header)
- Response follows same rules
 - first line: version, response code, response text
- Request Methods
 - GET
 - relevant data is in URL
 - form data, if needed, found in URL itself
 - often used in forms that search for data
 - POST
 - includes form input in message body
 - often used in forms that submit new data
 - HEAD
 - returns only header
 - check if modified
- Response codes
 - 2xx: success
 - 3xx: additional action required
 - 4xx: client problem
 - 5xx: server problem
- HTTP Connections
 - Non-persistent HTTP (version 1.0)
 - at most one object is sent over each TCP connection (one request, one response)
 - connection is closed as soon as data is transferred
 - web page with multiple objects require multiple connections
 - Suppose the web page we are interested in refers to another "object" that we need to fetch as well. How many round trips will it take to retrieve the web page and this additional object?
 - Response time consideration
 - for each object: one RTT to initiate connection, one

- RTT (plus transmission) to request each object
 - 2 RTTs per object
 - can be improved if several connections are opened in parallel
- Persistent HTTP (version 1.1)
 - multiple requests can be sent over single connection
 - responses are received in the same order as the requests
 - Response time consideration
 - RTT to initiate connection only once
 - one RTT (plus transmission) to request each object
- Pipelining
 - client may send several requests without waiting for response
 - Response time consideration
 - 1 RTT for connection
 - 1 RTT for first request (base request)
 - 1 RTT for all images
 - works well only when all requests to same server
- Cookies: per-user State
 - Web sites may use Cookies to maintain state
 - "remember me" authentication
 - session state
 - shopping carts, recommendations
 - process
 - response includes Set-Cookie header
 - browser saves cookie associated with the server
 - next request to the same server includes Cookie header with the same value
 - Two styles
 - all the state is in the Cookie
 - has to be one line, there are limits
 - total header size limited by server: $\sim 8\text{Kbytes}$
 - individual cookie size limited by browsers: $\sim 4\text{Kbytes}$

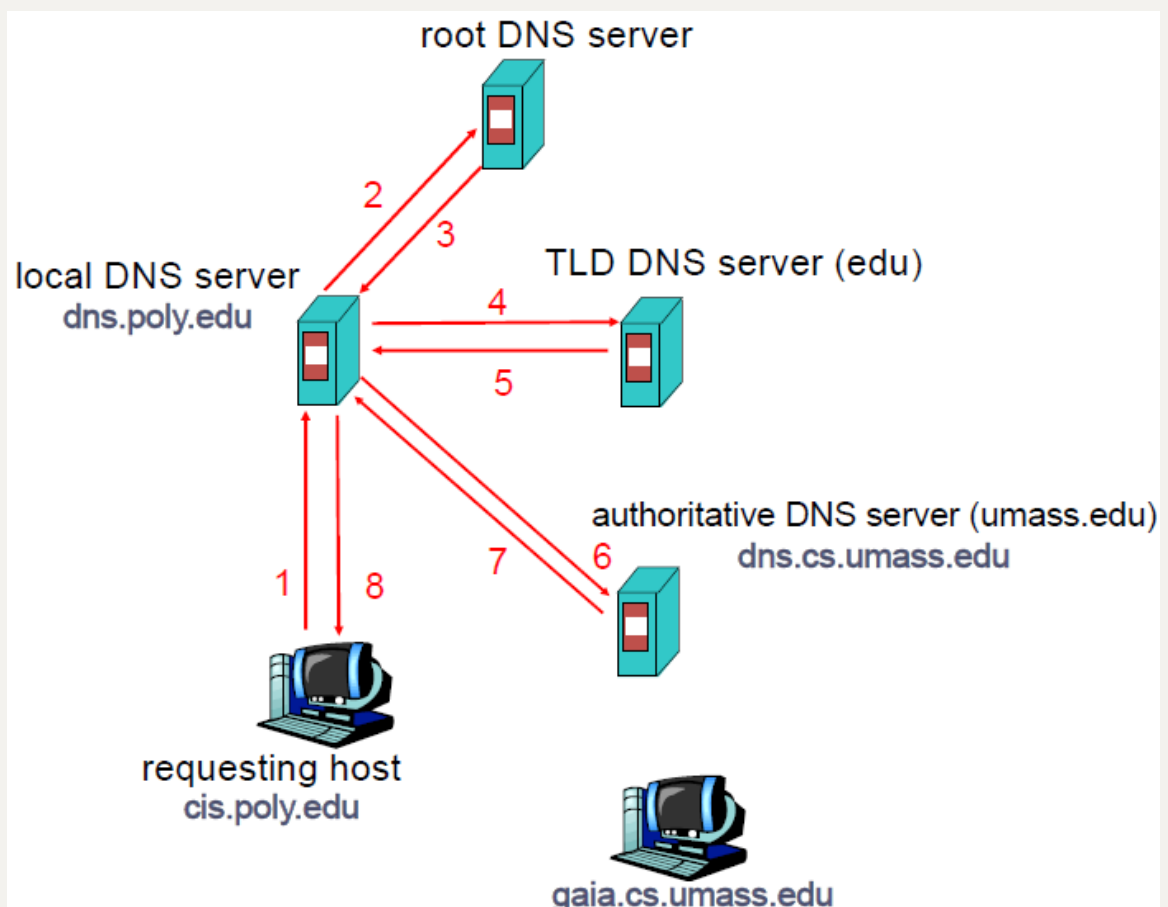
- State (or part of it) may be stored server-side
 - Cookie is used to identify entry in server database
 - Cookie can be just an email address or user identifier
- Web cache
 - browser may maintain a cached version of page
 - reduces network utilization
 - reduces page load time
 - cache may also be maintained in a separate host (proxy)
 - browser sends request to proxy, which redirects to server
 - if proxy has page cached, return immediately
 - typically closer to user than the server (company, ISP)
 - Web serves can provide cache policy
 - how long a page should be cached for
 - if cache is private (specific to user) or shared
 - conditional GET: request page, but only if modified

Sep. 19th - Module 3.3: DNS

- Naming and Network structure
 - Map user-friendly names to IP addresses
- Domain name
 - An identification string that defines a realm of administrative autonomy, authority, or control within the Internet
 - `www.example.com`
 - `www`: World Wide Web
 - `example`: Domain name
 - `com`: top level domain
- What is DNS
 - a **distributed database** implemented by a hierarchy of many name servers
 - an **application-layer protocol** used by hosts to communicate with name servers to **resolve** names
- Goals (design challenges)

- Scale (names, users, updates): 364.6 million domain name registrations
- ease of management (uniqueness of names)
 - who decides if `cs.ubc.ca` can name a host "students"
- availability and consistency and security
 - is there only one answer for the question "what is the IP address of `www.cs.ubc.ca`"
- performance
 - OpenDNS, Cloudflare, Google each serve > 1 trillion requests per day
- Solutions
 - Hierarchical design, caching, replication
- A hierarchy of name servers
 - top-level domain server: `ca, com, org`
 - authoritative name server: `ubc.ca, sfu.ca, ibm.com`
 - lower-level: `cs.ubc.ca, ece.ubc.ca`
 - individual machine: `students.cs.ubc.ca`
 - Local DNS server (cached data)
- Root name servers - 13 servers
 - 10 in the US, 2 in Europe, 1 in Japan
- TLD and Authoritative servers
 - Top-level domain servers: responsible for domains ending in
 - `.com, .org, .net, .edu` etc
 - all top-level country domains `.uk, .fr, .ca, .jp` etc
 - Authoritative DNS servers: provide authoritative hostname to IP mappings for an organization's subdomains and servers
 - can be maintained by organization and service providers
- Who knows what
 - every server knows the address of the root name servers
 - root servers know the address of all TLD servers
 - every node knows the addresses of its children
 - An **authoritative** DNS server stores name-to-address mappings (resource records) for all names in the domain that it has authority for

- each server stores only a subset of the total DNS database (scalable)
- Local name server
 - does not belong to the hierarchy
 - each ISP has one: also called "default name server"
 - when a host makes a DNS query, the query is sent to the local DNS server
 - acts as a proxy, forwards query into hierarchy of DNS servers
 - local DNS server resolves queries **iteratively**
- DNS lookups
 - Host at `cis.poly.edu` wants the IP address for the domain name `gaia.cs.umass.edu`
 - Host contacts local DNS server: `dns.poly.edu`
 - Local DNS server contacts the root server and subsequent DNS servers iteratively: `root`
 - each contacted server replies with name of next server to contact
 - authoritative server returns the IP address of the request domain
 - Local DNS server returns IP address to the host
 - the local DNS server remembers everything in the process

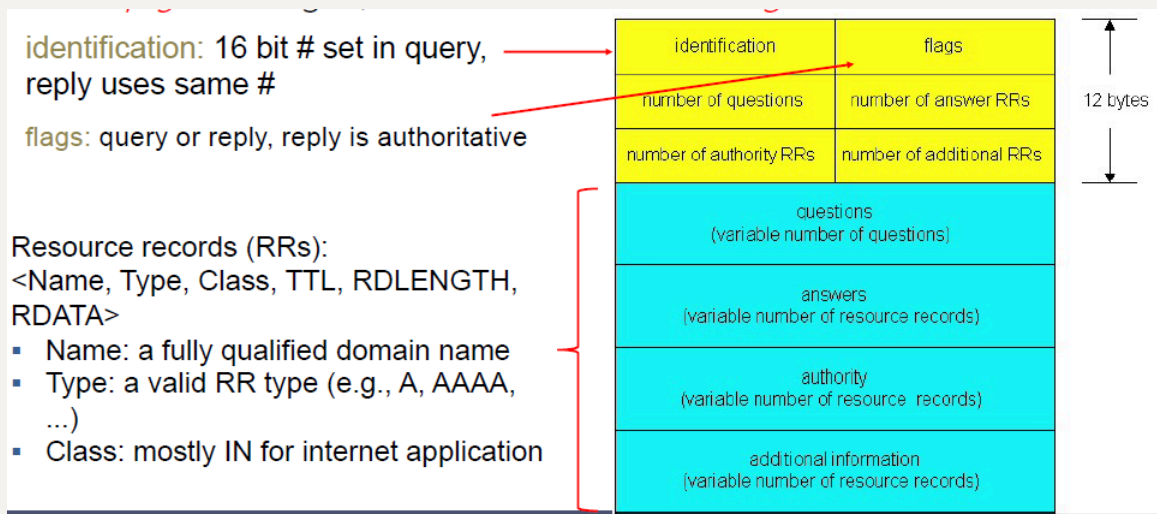


- DNS caching
 - local DNS servers cache response to queries
 - responses include a "time to live" (TTL) field
 - server deletes cached entry after TTL expires
 - it is effective because
 - top-level name servers very rarely change
 - popular sites are visited often → the local DNS server often has the information cached
- Inserting records into DNS
 - Register name **networkutopia.com** at a **registrar** (e.g. Network Solutions)
 - I provide registrar with names and IP addresses of my authoritative name servers (primary and secondary)
 - registrar inserts 2 RRs into the com TLD server for each authoritative name server
 - (networkutopia.com, dns1.networkutopia.com, NS)
 - (dns1.networkutopia.com, xxx.xxx.xxx.1, A)
 - (networkutopia.com, dns2.networkutopia.com, NS)
 - (dns2.networkutopia.com, xxx.xxx.xxx.2, A)
 - In the authoritative servers, add a Type A record for **www.networkutopia.com**
 - (www.networkutopia.com, xxx.xxx.xxx.100, A)
- DNS records
 - DNS servers store **resource records** (RRs)
 - RR is (name, type, value, TTL)

Type	Name	Value	Example
A (<u>A</u> ddress)	hostname	IPv4 address	(www.cs.ubc.ca, A, 142.103.6.5, TTL)
AAAA (<u>A</u> ddress x 4)	hostname	IPv6 address	(www.google.com, AAAA, 2607:f8b0:400a:80b::2004, TTL)
NS (<u>N</u> ame <u>S</u> erver)	domain	name of DNS server for domain	(cs.ubc.ca, NS, fs1.ugrad.cs.ubc.ca, TTL)
CNAME (<u>C</u> anonical <u>N</u> AME)	alias	canonical name	(foo.com, CNAME, relay1.bar.foo.com, TTL)

- DNS protocol

- Client-server interaction on UDP Port 53
 - message size limited by max UDP segment size (512 B)
 - for larger DNS messages, could use EDNS
 - spec supports TCP too, not always implemented
 - reliability via repeating requests if the client times out
- Query and Reply messages
 - both with the same message format
- Resolution is almost always "iterative"
- messages



- Questions are always **<name, type, class>** tuples
 - it is the only section included in a query
- Answers are RRs that match the tuple from the question
 - if a DNS server has CNAME pointers for the requested query with same class, returns CNAME records in the answer
 - may be multiple answers, since there may be multiple RRs with the same labels
- Authority RRs are type NS records pointing to name servers closer to the target name in the naming hierarchy
 - used to redirect the client to a "better" server
 - field is optional
- Additional RRs are record that the name server believes may be useful to the client
 - most commonly used to supply A or AAAA (address) records for the name servers listed in the Authority section

Sep. 22 - Module 3.4: E-mail

- Electronic mail
 - P.O. Box system
 - sender sends package to its local postal office
 - postal system forwards the package to final postal office
 - recipient must pick up package at its local office
 - email works the same way
 - replace "local postal office" with "local server"
- Components
 - User agents (UAs)
 - direct user interaction
 - compose, edit, read mail messages
 - Mail servers
 - maintain mailboxes (incoming messages) for each user
 - queue outgoing messages to be sent to other servers
- Protocols
 - Simple Mail Transfer Protocol (SMTP)
 - used to send messages from a user agent to a server (submission)
 - used to send messages from server to server (relay)
 - Post Office Protocol 3 (POP3)
 - used by a user agent to retrieve messages from a server
 - Internet Message Access Protocol (IMAP)
 - used by a user agent to retrieve messages from a server
 - Proprietary protocols
 - Microsoft exchange (MAPI, EAS)
 - Web interfaces
- SMTP (sending messages)
 - Uses TCP to transfer messages
 - port 25 for message relay
 - port 587 for message submission (other ports may be used if

this port is blocked)

- Command-response interaction (similar to DICT)
 - command: ASCII text, single line
 - response: 3-digit status code + text
 - In-band message transfer (ASCII only)
- Commands
 - HELO (EHLO), MAIL FROM:, RCPT TO:, DATA, QUIT
 - no commands for subject and other mail header fields
- Mail Message Format
 - Header lines
 - each line format - "name: value"
 - ends with blank line (end of header)
 - Message body
 - ASCII character only
 - may have multiple parts: attachment, text vs HTML
 - Binary data (attachments) are encoded using text-only format (e.g. Base64)
- Sending messages
 - Think of SMTP as manipulating an envelope
 - The SMTP commands instruct an SMTP server to send a message to particular receivers
 - The SMTP servers do not care about the content of the message
 - The message and the envelope can disagree
- POP3 (reading/retrieving messages)
 - TCP port 110
 - TCP port 995 for POP3S
 - Download and delete mode: client retrieves message and delete it from server
 - changing clients requires transferring messages from client to client
 - Download and keep mode: client leaves message in server
 - Stateless across sessions
- IMAP (reading/retrieving messages)

- TCP port 143 (993 for IMAPS)
- All messages are kept in the server and organized in folders
 - new messages stored in an INBOX folder
 - users can move messages into user-created folders
- message state, organization and deletion status is synchronized between server and all clients and across sessions
- Commands
 - create folders, move messages across folders
 - search remote folders based on matching criteria
 - can request for a part of the messages (e.g. just header)
- Protocol Bob uses to send email to his own server: SMTP or some other protocol
- Protocol Bob uses to read email from his server: POP3, IMAP, or some other protocol
- Performance
 - No one receives a large scale of emails per second.

Sep. 24th - Module 3.5: Peer-to-peer

- Bit-torrent
 - purpose: file sharing
 - initially designed in 2001, protocol v2 in 2017 (upgrading the hash function)
 - each node functions as both a consumer and provider of data
- File sharing scenario
 - suppose some N machines have a file (N might be just 1)
 - suppose some other M machines want the file (M might be very large)
 - if N machines share to all M machines, it might be very slow
 - $N = 1, M = 1000$, limited to the throughput possible by those N machines
- Examples
 - Gaming
 - Software updates (Android updates to billions of mobile phones)
- Bit-torrent

- All $N + M$ machines participate as both sources and sinks of data
- the N hosts are called "seeds"
- all hosts are called "peers"
- as soon as one of the M peers has a portion of the file it can share it with other peers
- send to 1000 peers
 - send $\frac{1}{1000}$ portion to each peer, each peer now shares its portion with the other 1000 peers.
- what if it is 1,000,000 clients $\rightarrow$ scales up similarly
- Details
 - the file is broken into many pieces
 - fixed size (except for the last one)
 - protected by a cryptographic hash (allows reliable detection of corruption)
 - a summary (torrent file) gives the necessary start up information
 - how many pieces
 - the hash of each piece
 - somewhere to start looking for peers (seeds, trackers)
- Basic operation
 - finding other peers
 - seeds or tracker to start with
 - peer exchange: each peer tells the other peers it is talking to regarding the peers it knows about
 - the group of peers for one particular file is a "swarm"
 - each peer talks to some subset of the swarm at any time
 - finding pieces
 - each peer shares the identity of the pieces it has with the peers it is talking to
 - a peer who does not have a piece asks a peer who has to share it
- Policy question

- which piece first?
 - rarest first, increase the overall "health" of a file
- which of all the peers in a swarm should a peer send data to?
 - the ones that are sending the most data to it (preferred peers)
 - tit-for-tat
 - random "opportunistic unchoking"
- Implementation
 - Bit-torrent is an open protocol with many implementations
 - Most use TCP as the transport mechanism
 - Some use μ TP - a UTP-based reliable transport protocol
- Thoughts
 - popular files can be found and obtained very quickly, unpopular files can be hard to find
 - often used to copy copyrighted material, but not exclusively
- Blockchain
 - purpose: unmodifiable transaction history
 - initially designed in 2008 by pseudonym Satoshi Nakamoto
 - uses cryptography to ensure that records added to the history can never be changed or removed
 - foundation for cryptocurrencies
 - centralized or de-centralized
 - primary value of the de-centralized approach is that it does not require a trusted agent
 - group of peers collaborate to decide which next "link" in the chain is accepted
 - Mechanism
 - all changes are broadcast to every other peer
 - changes are grouped into blocks
 - when a block fills up, it is added to the chain
 - there is no central repository of the "truth"
 - every peer holds all of the history
 - this makes it very hard to forget the history since it is

replicated very, very heavily

- History "forks"
 - Can have disagreement over whether newly added blocks are actually in the chain (concurrent modification)
 - group of peers eventually agree on the history
 - proof of work
 - each peer must demonstrate how much they like a particular version of the history by doing lots of work to support that version
 - peers are rewarded for doing this (mining)
 - the likelihood of a block being removed from the history gets progressively (exponentially) smaller as it gets older
- Bitcoin
 - tens of thousands active computers in the bitcoin peer-to-peer
 - anyone can join (initial peers found via DNS)
 - communication is built on top of TCP
 - Storage: current 600 GB

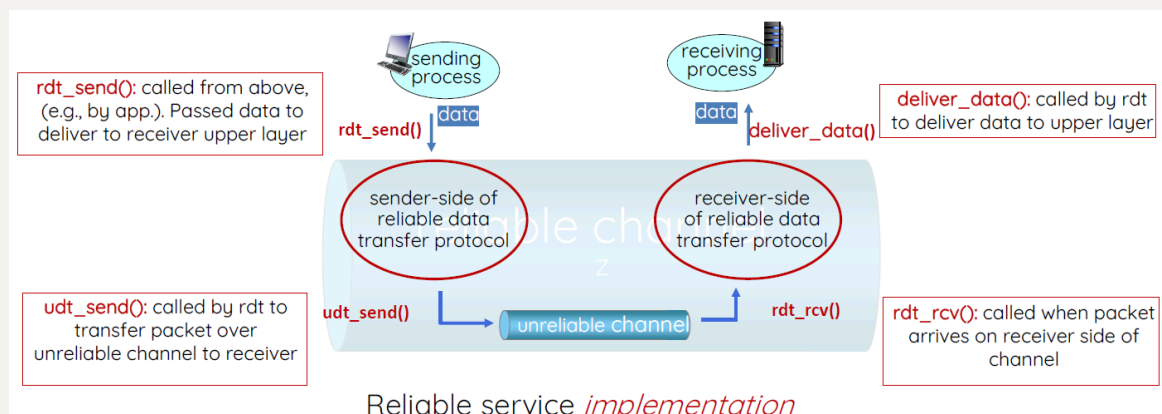
Sep. 26th - Module 4.1: Transport Layer - Introduction and UDP

- Purposes
 - provide **logical communication** between application processes running on different hosts
 - **multiplexing** of communication to different applications on end hosts
 - provide services to applications
- Logical End-to-end Communication
 - Network layer: **between hosts** (terminated at interface) - IP Address
 - Transport layer: **between processes** (terminated at application) - Port number
- Multiplexing applications over transport
 - an application is identified by a transport layer address: **<IP address, port>**
 - IP address: gets to the host
 - Port number: gets to some application process or thread on that host

- Historically 16-bit unsigned number
- Segmentation and Multiplexing & Demultiplexing and Reassembly
 - Transport protocols run in end systems
 - send side: breaks app messages into **segments** (TCP), passes to network layer
 - recv side: reassembles segments into messages (TCP), passes to app layer
- UDP - User Datagram Protocol
 - Simple and Fast, but Unreliable, no order guarantee
 - Segment format
 - Add source port number and destination port number
 - Add length of UDP segment + header
 - Add checksum, for error detection
 - Message after
 - Checksums
 - appear at transport, network, link layer
 - serve a different purpose at each layer
 - same algorithm at transport and network layer
 - Algorithm
 - treat the data as a sequence of 16-bit integers
 - function: addition of all these 16 bit integers (1's complement sum, carry out added back in)
 - checksum: 1's complement of the computed value (flip all the bits)
 - verifying by computing the same function (correct if 0)
 - Communication in Java
 - Socket class - **DatagramSocket**
 - Message class - **DatagramPacket**

Sep. 29th - Module 4.2: State Machines and Reliability

- Protocols
 - format and order of messages
 - actions to be taken on events
 - must provide a proper action for any event in any state
- Principles of reliable Data Transfer
 - Reliable service is an **abstraction**, it essentially relies on Network layer unreliable channels



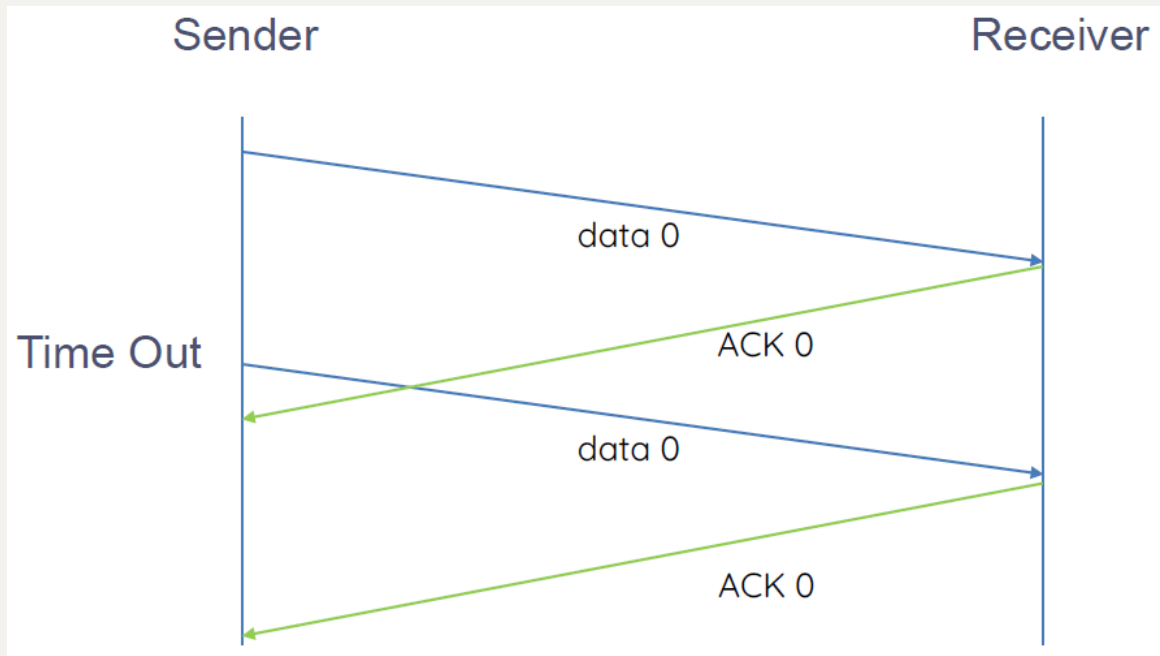
- Sender does not know receiver's side
- What could go wrong?
 - corrupted, lost, reordered, duplicate segments, etc
- Building a reliable protocol (v 0)
 - send only one segment at a time
 - identify when sending is allowable action
 - enumerate events and actions for both sender and receiver
 - draw state machine
 - initial assumption
 - one sender, one receiver
 - data to send comes from upper layer
 - nothing fails
- v 1
 - segments are never lost, but may be corrupted (needs **identify when re-sending is required**)
 - to ensure receiver receiving, needs an ACK or receipt
 - Possible events and actions

- Sender: data ready to send, send data
 - Receiver: segment received without problem, send ACK and deliver data
 - received is corrupted, send NAK
 - S: Receive ACK, ready to send more data
 - S: receive NAK, send data again
- v 2
 - what if ACK/NAK might be corrupted?
 - sender cannot be sure if the receiver the previous segment
 - what can sender do when sender received a corrupted ACK/NAK?
 - ignore the corrupt ACK/NAK - everything stops
 - treat it as an ACK - receiver may not have properly received the data
 - treat it as a NAK - receiver receive duplicated segments
 - Dealing with duplicate data?
 - Possible situations
 - S: data ready to send, receive ACK 0, receive ACK 1, receive NAK, receive corrupt response
 - R: segment 0 received, segment 1 received, segment received corrupted
- Complete FSM means every event is handled in every state

Oct. 1st - Module 4.3: Reliable Data Transfer - Lost segments and Timeouts

- v 3
 - Sender is responsible to detect both lost ACK and lost packet
 - No changes on the receiver
 - Corrupt or lost
 - if something is corrupt, we do not know what is corrupt
 - headers may have been corrupted
 - the corrupted segment may not even be designated for the receiver

- this case, corrupt data become lost data
- It is possible receive an ACK at Idle



- How long should the timeout be?
 - Too long - detect lost packet too slowly
 - Too short - resending packet too frequently
 - Measurable metric? RTT used to infer a proper timeout.
 - Assuming measured RTT of t , estimated RTT

$$ERTT_i = (1 - \alpha)ERTT_{i-1} + \alpha \times t$$
 where suggested $\alpha = 0.125$
 - but this can be very limited when the RTT has high variance
 - Include jitter (deviation of RTT)

$$\Delta RTT_i = (1 - \beta) \times \Delta RTT_{i-1} + \beta \times |t - ERTT_{i-1}|$$
 suggested $\beta = 0.25$
 - Timeout = $ERTT_i + 4 \times \Delta RTT_i$

Oct. 3rd - Module 4.4: Windowing Protocols

- Addressing Alternating Bit Protocol's Performance Problem
 - Sending multiple packets all at once
 - No finite state machine - too many states
- Windowing protocols

- sender sends many segments before waiting for an ACK (how many?)
- expand sequence numbers to integers to account for multiple in-transit and unacknowledged packets
- any or all of these segments might get lost
- sender has to be ready to resend any segments that get lost (buffer in-transit segments until acknowledged)
- receiver has to be ready to handle segments arriving out of order (buffer segments received out of order under "gaps" filled)
- Sender's window
 - range of segments that are stored for potential re-send
 - usually considered to be fixed size
 - window only moves when the first segment in the window is acknowledged
 - new segments are sent only when they "fit" in the window
- Receiver's window
 - store segments received out of order
 - out of order because of drops or re-ordering in the network
 - if window is 1 segment, out of order segments are dropped
 - once missing segment arrive, window is processed
 - segments received beyond window will be discarded
- Go-Back-N strategy
 - Receiver
 - window of size 1
 - when a segment is received, send an ACK for the last segment that was received in order
 - deliver the arriving segment if received in order, otherwise discard the segment
 - Sender
 - determines the number of outstanding (unacknowledged), segments held in memory
 - start timer on first segment sent
 - received ACKs may be cumulative (re-start timer on receipt)
 - On timeout go to last unACKed segment and re-send

everything (re-start timer)

- Sequence Number Range
 - limited by n , the number of bits used to represent them
 - sequence number arithmetic is modulo 2^n
 - the maximum sender window size for the range 0 to $2^n - 1$
 - Rule: sender's window size + receiver's window size $\leq$ sequence number range
 - So maximum sender's window size is $2^n - 1$

Oct. 6th - Module 4.5: Selective Repeat Protocol

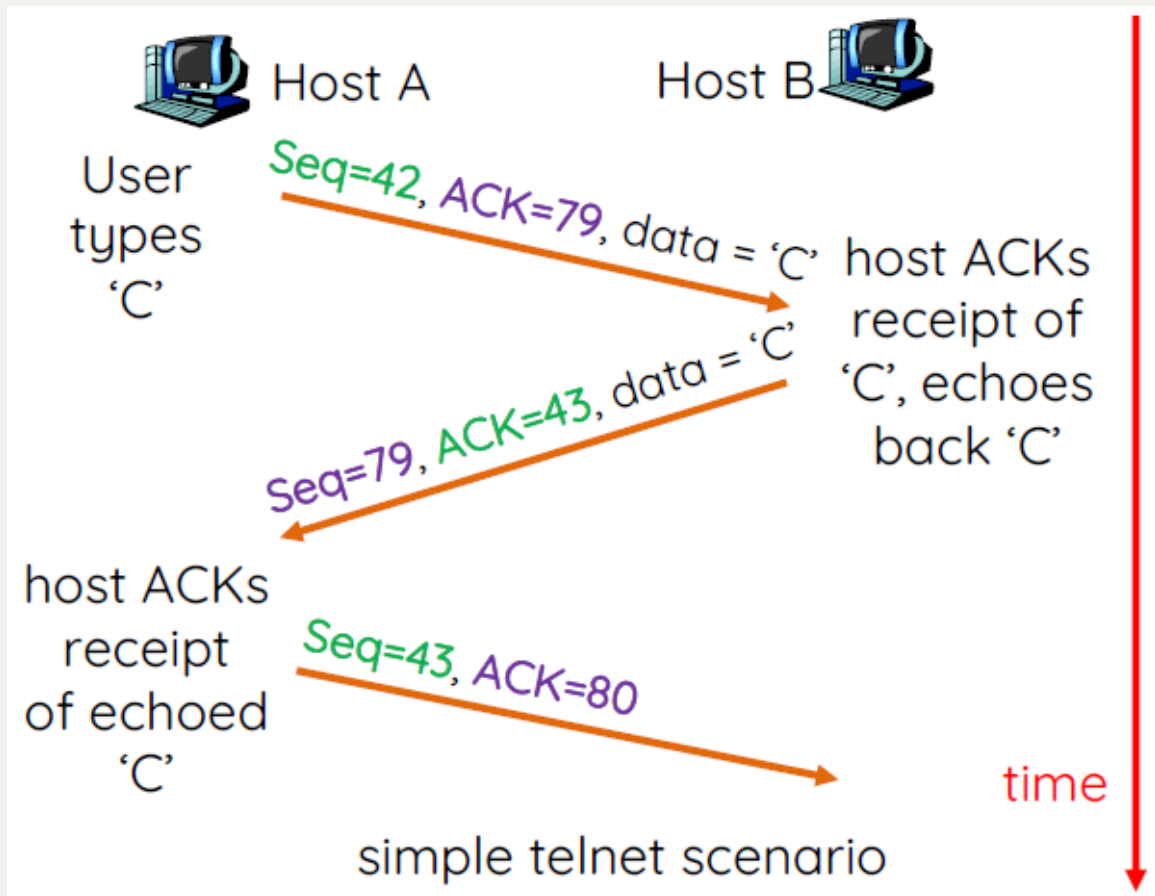
- Selective Repeat
 - Receiver
 - each segment is ack'ed individually
 - out of order segments are stored for later: receiver's window
 - Sender
 - can have a specific number of outstanding (unacknowledged) segments in memory: sender's window
 - each segment has its own timer
 - each segment is individually resent if timeout is reached
 - ACKs received in order move the sender's window
 - Sequence number
 - assume that segments cannot be re-ordered in the network
 - still: sender window size + receiver window size $\leq$ sequence number range
 - if we want sender window size = receiver window size
 - use $\text{floor}(\frac{n}{2})$
 - What if segment can be re-ordered in the network?
 - choose a large enough sequence number range
 - Window sizes
 - impact of larger sender window size
 - overwhelm the network and receiver
 - impact of larger receiver window size

- wasting network resource

Oct. 8th - Module 4.6: Flow and Congestion Control

- Flow control
 - send full window size?
 - receiver is slow in accepting data?
 - packets accumulate in the buffer, and packets will eventually be dropped
 - immediately re-sending packets will not solve the problem
 - receiver will notify the sender how much data it can handle
 - this information is usually included in the ACK
 - sender adjusts its window size based on this information
- Congestion control
 - network may be very busy
 - may result in packet loss, but it's the router that decides, sender needs to detect this loss
 - sender needs to slow down
 - missing ACKs are a sign there is congestion somewhere
 - if all packets are ACKed, we can increase the window again
 - use the rate of received ACKs out of expected ACKs to dynamically change window size
- Effect on throughput
 - throughput affected by
 - bandwidth of sender's direct network connection
 - receiver's specified window size (flow control)
 - sender's specified window size (congestion control)
 - if sender's direct connection is not the bottleneck
 - another router will experience congestion elsewhere
 - congestion control will reduce transmission speed
- TCP: Overview
 - a bunch of RFCs

- Point-to-point (exactly two participants)
- connection-oriented
 - handshaking, initializes sender, receiver state before data exchange
- full duplex data
 - bi-directional data flow in same connection
- send and receive buffers
- reliable, in-order *byte stream*
 - no "message boundaries"
 - MSS: maximum segment size
- pipelined
 - TCP congestion and flow control set window size
- flow controlled
 - sender will not overwhelm receiver
- TCP sequence numbers and ACKs
 - each byte of a TCP segment has a sequence number
 - the sequence number of a segment is the sequence number of its first byte
 - ACKs correspond to the first sequence number not yet received
 - similar to Go-Back-N, but byte-based and +1
 - receiver stores packets in its own window
 - like Selective Repeat
 - ACKs are cumulative
 - like Go-Back-N



- TCP window management
 - receiver has a flow control window
 - size measured in bytes and is selected by the application
 - actual send window is the minimum of the flow control window (advertised by the receiver) and the (scaled) congestion window (computed by the sender)
 - sender has a congestion control window
 - size measured in segments
 - determined by the presence or absence of congestion

Oct. 9th - Module 4.7: TCP 1

- TCP Retransmission
 - based on timeout
 - a timer is set for the first unacknowledged segment
 - when the timer expires we retransmit
 - retransmit just one segment
 - fast retransmission

- four or more ACKs with same ack number trigger a retransmission without a timeout
 - triple duplicate ACK
- Congestion detection
 - congestion detected via
 - timeouts, or multiple identical ACKs
 - sender adapts its congestion window
 - lack of congestion is detected via
 - sending a window full of data without detecting congestion
- Congestion management
 - TCP is conservative, it never wants to overwhelm the network with data
 - at first sign of congestion it slows down a lot
 - cuts congestion window in half
 - multiplicative decrease
 - when it appears that congestion has eased it significantly
 - increase congestion window one by one
 - linear increase
- Slow start
 - we start with a congestion window of 1 segment
 - increase by 1 each time a segment is ACKed
 - equivalent to doubling it each time we send a window full of data with no congestion detected
 - stop doubling when we detect congestion
- Evolving congestion management
 - Tahoe
 - very aggressive in response to congestion
 - drop the congestion window to 1 and enter slow start
 - Reno and New Reno
 - cut the congestion window in half
 - Vegas - congestion avoiding
 - use increasing RTTs as an early signal of congestion
 - slow down when RTTs increase

- speed up when RTTs decrease
- BIC
 - faster recovery from congestion
 - binary search between the congestion window and the max window size
- CUBIC
 - the congestion window is not changed every RTT
 - instead a cubic function of time since the last congestion detection

Oct. 15th - Module 4.8: TCP Connection Control, Wrapup

- TCP Connection Establishment
 - "three-way handshake" to establish connections
 - client sends an initial SYN message
 - initial sequence number for client → server is specified
 - server responds with a SYN/ACK message
 - client → server sequence number is confirmed in ACK
 - server → client initial sequence number is specified
 - client sends an ACK message
 - server → client sequence number is confirmed in ACK
 - Opening a connection - client opens connection `connect()`
 - Step 1: client end system sends TCP SYN control segment to server with an initial sequence number (ISN)
 - Step 2: server receives SYN, replies with SYN/ACK with server ISN (server: `listen()`)
 - Step 3: client receives SYN/ACK, replies with ACK and may include data
- Single Initial Sequence Number Problem
 - it is possible that a delayed sequence is received at the just right time when the server is expecting another sequence with the same sequence number
 - server can treat the delayed sequence as the "valid" sequence, while discarding the actual valid sequence

- solution: do not use 0 as the initial sequence number **all the time**, choose an arbitrary number
- TCP Connection Termination
 - The side that wants to terminate sends FIN message (FIN means "I will not send any more **new** data in this conversation", packet can still be **resent**)
 - other side responds with ACK
 - The other side will also send a FIN message
 - it may not send it immediately, since it may have more data to send
 - the peer also responds with an ACK
 - Alternative: connection reset (RST message)
 - usually used if other side misbehaves or if disconnection or too many timeouts are detected
 - Closing a connection - client closes connection
 - Step 1: client end system sends TCP FIN control segment to server (**close()**)
 - Step 2: server receives FIN, replies with ACK. Some time later, it closes connection, sends FIN (server: **close()**).
 - Step 3: after servers sends FIN, and client sends ACK. If this ACK keeps getting lost, after the "grace period" for client, it will send RST.
 - Step 4: server receives ACK, connection closed.
- Complications
 - slow start congestion window details - ssthresh (slow start threshold)
 - Nagle's algorithm - delayed sends
 - delayed ACKs - only send half the ACKs
 - "Silly Window Syndrome"

Oct. 17th - Module 4.9: Alternative Transport Protocols

- How do we achieve reliability?
 - Checksums, SEQ and ACK numbers (ordering, reliable delivery), timeouts and retransmissions (loss, delays, congestions), Checksum (data corruption)

- Some other transport protocols
 - DCCP - Datagram Congestion Control Protocol
 - does not provide reliable in-order delivery for data
 - provides application-selected congestion control
 - useful for multimedia streaming, online gaming
 - SCTP - Stream Control Transmission Protocol
 - message-based like UDP
 - multiplex several application "streams"
 - independent reliable in-order delivery and congestion control for each stream (like TCP)
 - used for VoIP
- Majority of Today's Internet Traffic
 - HTTP + TLS + TCP (lot of traffic suffering from inefficiencies in this protocol stack)
 - Problem with TCP
 - 3-way handshake too expensive
 - worse with added TLS handshake for HTTPS
 - RTT estimation too inefficient
 - Sequence numbers conflate ordering and reliable delivery
 - retransmitted segments have the same sequence number as the original
 - confounds RTT estimates
 - Problem with HTTP 1.1 over TCP
 - Overheads: ASCII headers - "content-length" = 15 bytes = 120 bits
 - Suffer from head-of-the-line blocking (over TCP)
 - a small request blocked behind a large request
 - loss of any packet prevents all transfers
- Improvements in Application Protocol
 - HTTP/2
 - headers are encoded in binary and compressed

- compressed binary format called QPACK
 - Limitation: not human readable
- server allowed to push objects without being asked
 - via "push promises" - reduce latency for objects client likely to request
 - client can say "no thanks, I got it already"
- responses may be reordered with respect to requests
 - limitation: can still prevent transfers if TCP loses packets
- Required improvements
 - multiplex requests independently (avoid HOL blocking)
 - improve connection handshake protocol
 - reduce congestion control inefficiencies
- Layered Architecture creates constraints
 - may need to change kernel on all devices
 - upper layers cannot control lower layers
 - Idea: co-design application and transport protocols (improve performance by combining some of the protocols)
- Alternative: QUIC
 - Quick UDP Internet Connections
 - Add reliability on top of UDP
 - implemented in user space rather than in the kernel
 - Could be an alternative to UDP and TCP above IP layer itself
 - problem: existing network middleboxes cannot handle new protocols
 - Key features
 - integrates TLS into the transport
 - complete both TCP and TLS handshake in one RTT
 - QUIC handshake

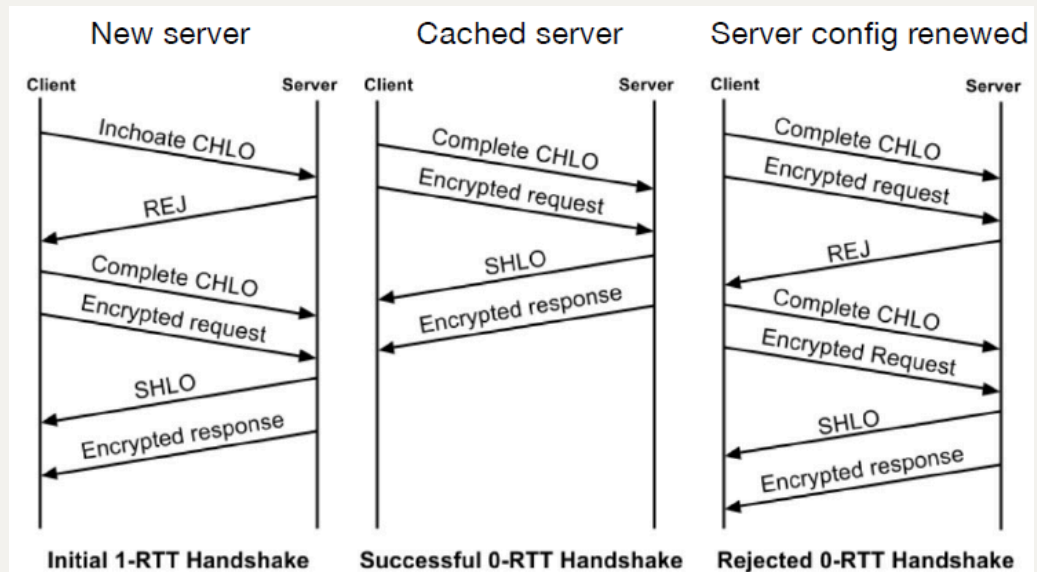


Figure 4: Timeline of QUIC's initial 1-RTT handshake, a subsequent successful 0-RTT handshake, and a failed 0-RTT handshake.

- Key features
 - multiple streams within a single connection
 - handle losses, flow control in each stream independently
 - avoid HOL blocking
 - priorities can be set per stream
 - streams are very light-weight
 - using a new stream identifier creates the stream
 - HTTP/3 uses a different stream for each object
 - separate sequence numbers for data and frame delivery
 - decouple reliability and ordered delivery
 - better RTT estimation
- Packet numbers
 - used for RTT estimation and congestion detection
 - every packet gets a new one, including retransmitted ones
 - starts at 0
 - it is okay to start at 0 all the time, because packets with same packet numbers are encrypted using different keys
- Stream offsets
 - used for reliable delivery, like the TCP sequence number, start at 0

- Key features
 - mobility
 - can migrate QUIC connections from one device to another
 - the client-selected connection ID is the key, not the client's host address
 - e.g. from Wi-Fi to cellular

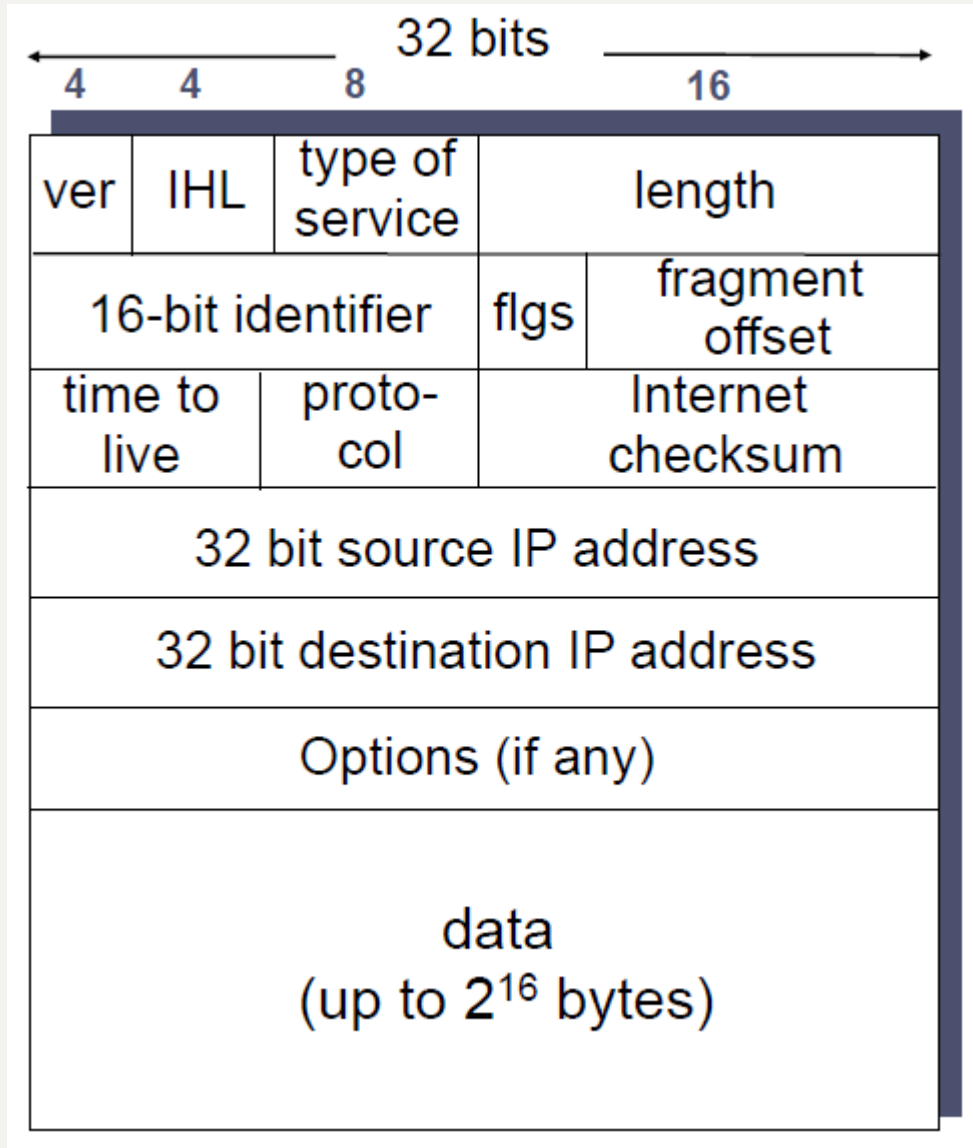
Oct. 20th - Module 5.1: Network Layer - History and Autonomous Systems

- TCP/IP hourglass
 - IP sends packets (Datagrams) from source to destination
 - IP is the "waist" of the Internet
- Beginning...
 - imagine a bunch of private networks
 - NSFnet: connected U.S. universities
 - CAnet: Canadian internet
- Internet Organization Structure
 - organized into entities called autonomous systems (ASes)
 - remnant of old private networks
 - each autonomous system
 - is assigned a range/collection of IP addresses
 - is responsible for routing to address it "owns"
 - is responsible for routing to addresses that are not its responsibility
- Who controls an AS
 - some companies do the end systems (last mile)
 - end users, incoming traffic are only directed to clients within the system
 - some companies do long haul lines (backbones)
 - within a country or region, around the world
 - some do a mix of both

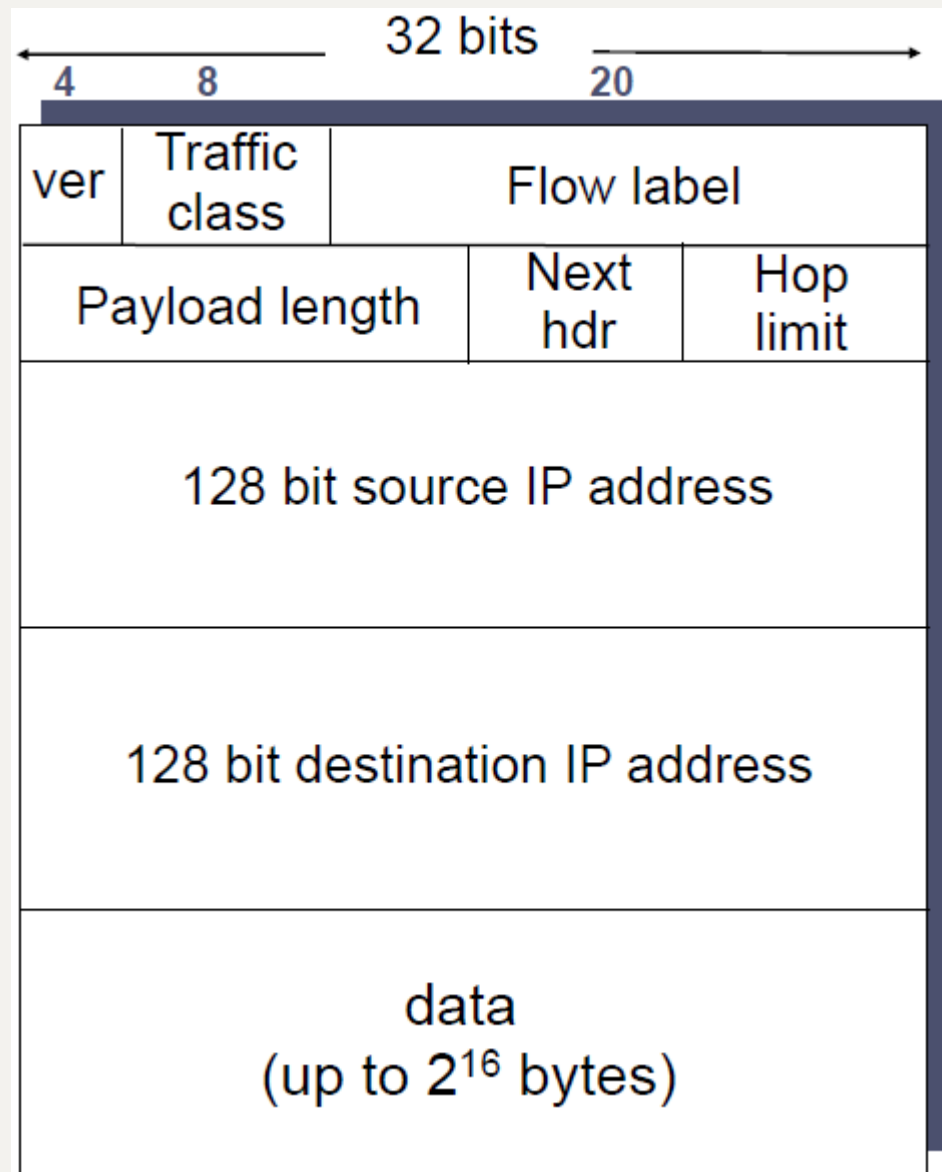
- some companies are consortiums
- networks often connect to each other at Internet Exchanges (belong between autonomous systems)
 - PoP: Point of presence (belongs to a certain autonomous system)
- Tier 1 Networks
 - do not pay other autonomous systems for delivering messages
 - can reach whatever network around the world
- Tier 2 Networks
 - still need to pay something to transmit data over the Internet, cannot reach all the networks
- Tier 3 Networks (STUBS)
 - everybody else, e.g. UBC system
- How does the data move?
 - we need to think about names (or addresses), and deciding where to send the data
- Naming
 - Format, semantics, properties
 - Addresses name **Interfaces** NOT **Hosts**
 - e.g. Wi-Fi is an interface, Ethernet is an interface, and also consider loopback (`localhost`)
- Why name interfaces?
 - the address has to identify the interface, as you expect the data to come back on the same interface
- What exactly is a "network"?
 - we talked about ASes as networks - a collection of computers and routers owned and administered by a single organization
 - when we talk about forwarding, we talk about a network as a single shared communication medium connected to at least 2 interfaces

Oct. 22nd - Module 5.2: IP Addresses and Forwarding

- IP service model
 - best effort delivery from one network interface to another network interface
 - no guarantees
 - no (per flow) state in the routers
 - split into two parts: forwarding, routing
- Two key network-layer functions
 - forwarding: move packets from router's input to appropriate router output
 - getting through a single intersection
 - routing: determine route taken by packets from source to destination
 - planning the trip from source to destination
- Network layer components
 - Data plane - local, per router function
 - local to one router
 - forwards datagrams based on forwarding table
 - Control plane - network-wide logic
 - global across all routers
 - decides how to route datagrams and updates forwarding table
 - traditional routing algorithms (implemented in routers)
 - software-defined networking (implemented in (remote) servers)
- IPv4 Datagram Header



- words: 32 bits; byte: 8 bits; nibble: 4 bits
- IPv6 Datagram Header



- IP Addresses
 - IPv4 original address - obsolete
 - IPv4 class-based addresses - obsolete
 - IPv4 class-less (CIDR) addresses
 - IPv6 addresses
- IPv4 address
 - 172.16.254.1
 - 1 byte = 1 octet = 8 bits
 - originally, 8 bit network number, 24 bit host number within the network
 - The first and last address are never used for an actual host
 - 17.0.0.0: usually used to represent the network and for hosts that don't have an address yet
 - 17.255.255.255: used for broadcast (sending a datagram to all

the hosts in the network)

- Class-based addresses

Prefix	Network Size (bits)	Name
0	8	Class A
10 (128 – 191)	16	Class B
110 (192 – 223)	24	Class C
1110 (224 – 239)	32	Class D - multicast
1111 (240 – 255)	32	Class E (reserved)

- e.g. 192.168.10.52 is a Class C address → the network address is 192.168.10, the host number is 52
- Classless inter-domain routing addresses
 - allow any number of bits in the network number
 - the rest of the bits name the host within the network
 - -IP_address/number_of_fixed_bits
 - alternative format: netmask
 - $200.23.18.4/22 = 200.23.18.4/255.255.252.0$
- IPv6 format
 - 128 bits, 32 hex-digits (8 hextets) separated by colons
 - zero suppression
 - separated into network+subnet (48 + 16) and host (64)

Oct. 24th - Module 5.3: IP Addresses Ranges

- What operations do we need on IPs?
 - in the same network?
 - if yes, no need to go through router
 - if not, must go through a router
 - which of many networks is an IP address in?
 - forwarding purpose
- Forwarding Table
 - each router keeps a forwarding table indicating, for each destination IP range, the link to be used

- formally, the Forwarding Information Base
- built from aggregating information retrieved from other routers (routing table, or RIB)
- Longest prefix matching
 - if more than one range in the table matches an IP address, the most specific entry is used
 - since ranges are prefixes, the longest prefix is the most specific
- Non-routable IP addresses
 - some addresses are defined to be non-routable
 - if a router sees a datagram with a non-routable destination address, it just throws it away
 - RFC 1618 defines these 3 CIDR networks to be non-routable
 - 10.0.0.0/8
 - 172.16.0.0/12
 - 192.168.0.0/16
- IP address assignment
 - are assigned to ISPs through the Regional Internet Registries
 - then to various ISPs
 - who assigns sub-portions to their customer ISPs
- Address Range Aggregation
 - ranges can be aggregated if, when combined
 - all ranges being aggregated are covered
 - new range does not include groups not being aggregated
 - useful for collapsing range information
- Address Range Division
 - first consider dividing into 2 sub-ranges
 - set the first unfixed bit as a fixed bit in the two sub-ranges
 - one range has that bit as 0, the other has that bit as 1
 - repeat if multiple ranges are required

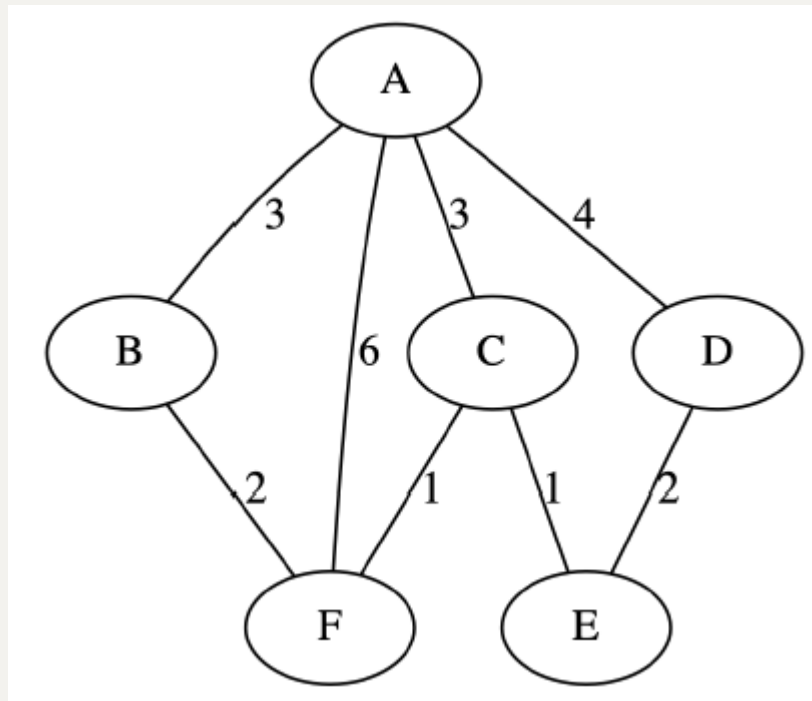
Oct. 27th - Module 5.4: Routing

- Key challenge for forwarding
 - Imagine a 32-port (interface) router, each interface has 40 Gbps bandwidth, average datagram is 500 bytes
 - how long does it have to forward each datagram? 320×10^6 datagrams per second or 3 ns per datagram
- Routing protocol
 - Goal: determine "good" paths, from a sending host to a receiving host, through a network of routers
 - path: sequence of routers that packets will traverse in going from given initial source host to given final destination host
 - "good": least "cost", "fastest", "least congested"
- Alternatives for deciding routing
 - configured statically by network operators
 - dynamically by having the routers exchange messages
 - construct an entire network map
 - distributed and decentralized
 - on failure we want to converge to the same view of the network
 - configured centrally by software (Software Defined Networking)
- Routing (two levels)
 - Routing within a single AS, under the single control of a single administrative entity
 - Interior Gateway Protocols - IGP
 - Routing between different AS, where we have no control over the routing policies of the other AS
 - Exterior Gateway Protocols - EGP
- Interior Gateway Protocols
 - Link State
 - broadcast link information to all routers
 - create a "map" of the network
 - Distance Vector
 - send local information to neighbouring routers

- create "signposts" for routing
- Link State
 - each router tells every other router about all of its links
 - this gives every router complete information about the entire network
 - every so often, each router uses Dijkstra's algorithm to find the shortest path to all the other routers
 - it then updates its forwarding table
- Dijkstra's Algorithm
 - represent the network as a graph
 - nodes correspond to routers
 - networks are attached to routers, but not explicitly shown
 - label on the edge corresponds to the "cost" of using a link between routers
 - compute the lowest cost path from a single node in a graph to all other nodes
 - dynamic programming
 - Algorithm
 - initialize all costs to infinity and all prevs to undefined
 - for each neighbour of the source, enter cost and prev
 - loop until all nodes are in Done
 - choose a min cost node not in Done, call it X
 - Add X to Done
 - For all neighbours of X
 - if path through X is cheaper, update Cost and Prev
- How expensive is using a link-state routing algorithm?
 - Sending the message between all routers already takes: $O(N^2)$
 - Dijkstra's algorithm: $< O(N^2)$
 - the number of links in the network (L) is linear to the number of routers in the network (N)

Oct. 29th - Module 5.5: Distance Vector Routing

- Distance Vector



- every so often, each router (A) tells its neighbours about the cost of its best routes to the networks it knows about
- a receiving router (B) checks to see if any of these routes would shorten their path to the destination
- if so, it updates its forwarding table to forward through A
- A's view (each router only knows the cost to its neighbours, A won't know there is a connection between C and E)

	A	B	C	D	E	F
A	0	3	3	4	-	6
B	-	-	-	-	-	-
C	-	-	-	-	-	-
D	-	-	-	-	-	-
E	-	-	-	-	-	-
F	-	-	-	-	-	-

- assume each "-" to be a cost of infinity
- (???) In distance vector routing, each iteration lets information travel at least one hop farther. So, convergence is guaranteed after a number of iterations equal to the graph's diameter.
- Link state just requires one round of broadcast among all nodes
- Forwarding loops
 - if the cost of a link decreases, nothing bad happens
 - if the cost of a link increases, forwarding loops can be created
 - creating a forwarding loop
 - A - C (3); A - F (6); C - F (1)
 - change to A - C (50). F used to tell C, can go to A with 5 cost (through C). C will forward to F, but F will then forward it back to C
- Avoiding loops - poisoned reverse
 - if F routes to A through C, it advertises to C that its cost for A is infinity
 - works for loops involving 2 routers
 - does not work for loops involving more than 2 routers
- OSPF (Open-Shortest-Path-First)
 - currently most used IGP in the Internet
 - A link state protocol

- supports extensions (areas - to support hierarchy, more scalable updates)
- runs over IP (protocol number 89)
- IS-IS (Intermediate System to Intermediate System)
 - IS is DEC-speak for "router"
 - Popular for very large ISPs
 - A link state protocol
 - scales well to very large networks
 - Uses Link Layer Frames rather than IP to send messages
- EIGRP (Enhanced Interior Gateway Routing Protocol) - Cisco
 - A distance vector protocol (a bit of a hybrid)
 - fast convergence times
 - less resource usage than OSPF
 - Uses IP protocol 88 and Cisco's Reliable Transport Protocol (RTP)
- RIP (Routing Information Protocol)
 - The original distance vector protocol
 - slow convergence times
 - only works for small networks
 - the only metric is hop count
 - uses IP broadcast
- Summary: IGP
 - do not scale to the scale of the Internet
 - do not account for administrative differences (administrative autonomy) (political, company, international boundaries)
 - do not allow policy to play a role

Oct. 31st - Module 5.6: Inter-Domain Routing

- Inter-domain routing
 - The Internet is organized into ASes, each AS is responsible for some collection of IP addresses, each AS must tell other ASes: which addresses it owns and which addresses it will route to
 - "right path" from UBC to UofC: UBC - BCnet - Canarie - Cybera - U

of Calgary

- Peering and Transit

- Peering: Two ISPs pass traffic between each other for their "customers"
- Transit: Passing traffic across an AS to get to a different AS
- Stub ASes: Have a single provider (or two) as their Internet Gateway (no reason for any transit traffic to go through a stub AS)

- The (mini) Internet

- ASes are interconnected with routers
- Internal routers: to route traffic in its own network (IGP)
- Border routers: to route traffic at the edge of ASes
- Edge routers: connect a home or company network to the rest of the AS (and the world)

- Inter-AS Routing Requirements

- discover reachable subnets (network prefixes) from neighboring ASes
- determine best routes to reachable subnets (prefixes)

- Exterior Gateway Protocol (EGP)

- Border Gateway Protocol
- executed by border routers (routers at the border between ASes)
- forms the backbones of the Internet, along with IP
- iBGP and eBGP
 - both are application layer protocols, built on top of TCP
 - BGP connection or session between routers running BGP
- between ASes, uses **aggregation** or **summarization** to reduce table sizes

- Obtaining prefix reachability information

- eBGP: obtain subnet reachability information from neighboring ASes
- iBGP: propagate reachability information to all AS-internal routers
 - the AS can also use some other IGP to do this

- eBGP - path vector routing

- advertise **reachability** information of a prefix x to other ASes
- path vector - variant of distance vector
- advertise prefixes
 - AS number, AS path (prevent looping by allowing receiver to

look for itself), next hop to take

- receiver of advertisement builds forwarding table
 - AS management decisions (policy), shortest route, closest connecting center, other factors
- Route aggregation
 - route summarization
 - reduces the size of the routing table
 - reduces the number of advertisements
- iBGP
 - border routers use iBGP to distribute info on routing to all internal routers
- Determining routes to subnets
 - hot potato routing: hand the packet off to the other AS as soon as possible
- Challenges with BGP
 - simple and globally consistent, but
 - policy is complex and locally determined (routes accepted, rejected, trusted, propagated based on local decisions)
 - greater flexibility for organizations
 - vulnerable to bogus route propagation
 - BGP route leaks, route hijacking, MITM (man-in-the-middle, "USA") attacks (e.g. some organization that "looks at traffic to gain information")
- Route hijacking
 - advertise a prefix as part of a different AS than it actually is
 - inadvertent - leak
 - malicious - hijack
 - if unused prefix - mildly bad behaviour
 - if prefix used in another AS - cause re-routing of traffic of the victim subnet and potential denial-of-service
- MITM attack
 - hijack a route from a source to a victim
 - re-route traffic via routers under adversarial control before forwarding it to the victim

- Mitigations
 - transit AS providers need to check for false advertisements and filter them
 - custom filtering is laborious and error-prone
 - need to build filters from various routing registries
- Software defined network
 - for intra-AS routing
 - an alternative to running routing algorithms in the routers
 - a centralized service makes all the routing decisions, then tells each router what to do
 - OpenFlow
 - used by a number of companies (Google, China Mobile)
 - centralized solutions are better than distributed ones

Nov. 3rd - Module 6.1: Network Address Translation (NAT)

- Worst network nasty trick
 - we should have run out of IP addresses
 - last addresses allocated by IANA on 31 January 2011
 - Regional Internet Registries allocated their last addresses across 2011 to 2019
- Two choices
 - actually switch to IPv6
 - play games with IPv4 addresses pretending that we have more than actually have
 - increasing complexity, ignoring layering, breaking abstraction, providing partial functionality
- Argument for 2nd choice
 - we now have run out of IPv4 addresses
 - do we really need a globally unique address for every device?
 - what would other hosts need your address for?
 - do other hosts initiate a connection to your host?
 - most connections target servers "out there" in the Internet

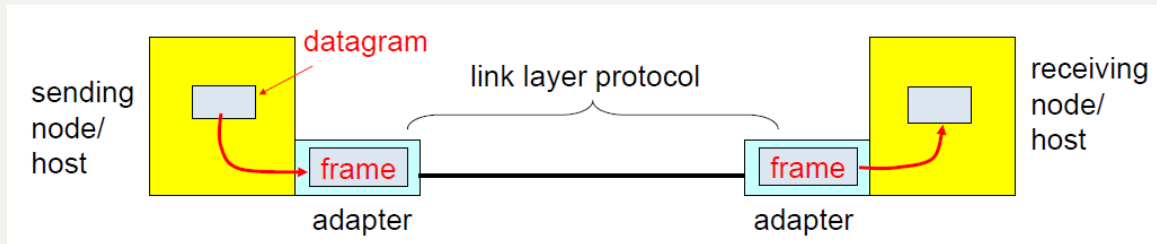
- maybe several hosts can share an IP address
- Non-routable IP addresses?
 - some addresses can never be advertised publically
 - initially used only for local communication (no Internet connection)
 - all the hosts in a network (your house) share a single IP address
 - the address is "owned" by the edge router (cable modem, etc)
 - the one that attaches to your ISP
 - all other hosts in the network (your house) get a private network address
 - we play games with IP addresses as datagrams pass through your edge router
- NAT
 - call the edge router "NAT router"
- Identifying a connection
 - `<source IP, source port, destination IP, destination port, protocol>`
 - protocol: transport, typically TCP or UDP
 - local and remote
 - incoming messages at a router: local is receiver, remote is sender
 - outgoing messages at a router: local is sender, remote is receiver
- Handling of Messages through NAT
 - internal client $\rightarrow$ external server
 - router converts source IP to router's public IP
 - router converts source port to some available port
 - router adds entry to NAT table describing this mapping
 - internal client $\leftarrow$ external server
 - router finds NAT table entry
 - router changes destination IP/port to value in the table entry
- Server on the inside
 - what if the initial connection comes from outside?

- server is inside the NAT, connection where both hosts are inside NATs
- NAT translation will not find an entry in the table
- Option 1: manually add a fixed rule - port forwarding
 - add a rule that if a connection comes to port **200.1.2.3:8080**, it should be forwarded to **192.168.0.2:80**
- Option 2: UPnP (universal plug and play)
 - protocol that runs on the router and internal host
 - host asks router to enter a forwarding rule
- Problems with NAT
 - design-level concerns
 - breaks layering, routers are not supposed to be modifying IP addresses, much less ports
 - security concerns
 - port forwarding/UPnP opens ports for anybody to connect to your device
 - expose to viruses and other attacks
 - some applications do not work well
- Applications and NAT
 - in some application-level protocols, clients advertise their IP address in their messages
 - internal host advertises address to be used for connection
 - FTP active mode; P2P applications
 - if behind NAT, how does client know which IP to use
 - depends on the application
 - FTP: use passive mode
 - P2P: typically require manual set up or UPnP

Nov. 5th - Module 7.1: Link Layer - Introduction, Error Detection

- Link Layer: Introduction
 - hosts and routers are nodes
 - communication channels that connect adjacent nodes along communication path are links

- wired links, wireless links, LANs
- link layer packet is a **frame**, encapsulates datagram (network layer packet)
- it has responsibility of transferring a datagram from one node to an adjacent node over a link (physical medium)
- Adaptors communicating



- link layer implemented in "adapter", aka Network Interface Card (NIC)
 - ethernet card, 802.11 card Wi-Fi
 - often built-in
 - one adapter on each side
 - adapter is semi-autonomous
 - link and physical layers
- Multiple Access Links and Protocols
 - two types of "links"
 - point-to-point
 - PPP for dial-up access
 - point-to-point link between two routers
 - broadcast (shared wire or medium)
 - traditional Ethernet
 - upstream fiber, coax, or Hybrid Fiber Coax
 - 802.11 wireless LAN
- Link Layer Services
 - Framing (or packetizing)
 - encapsulate datagram into frame, adding header, trailer
 - media access control "MAC" addresses used to identify **source, dest**
 - different from IP addresses
 - broadcast medium

- Link Access
 - MAC protocol (media access control)
 - with half duplex, multiple nodes on a link can transmit, but not at the same time
- Reliable delivery (retransmission)
 - seldom used on low bit error links (fiber, twisted pair)
 - more often used on wireless links: high error rates
- Error Detection and Correction
 - errors caused by signal attenuation, noise
 - receiver detects presence of errors and drops frame
 - receiver identifies **and corrects** bit error(s)
- Frame
 - Ethernet header: 14 bytes
 - **dest** and **source** MAC address: 6 bytes each
 - **EtherType**: 2 bytes
 - minimum transmission frame size: 64 bytes
 - Ethernet payload (data): 46 - 1500 bytes
- MAC address
 - 48 bits - 6 bytes, given in hexadecimal, locally administered
 - **88-B2-2F-54-1A-0F**
 - broadcast address is **FF-FF-FF-FF-FF-FF**
- MAC versus IP
 - 32-bit IP address
 - used to get datagram to destination IP subnet
 - must be unique in the world
 - 48-bit MAC (or LAN or physical or Ethernet) address
 - used to get frame from one interface to another physically-connected interface (same network)
 - MAC address (for most LANs) burned in the adapter ROM
 - Generally unique in the world, but must be unique in the network
- MAC assignment

- MAC address allocation administered by IEEE, manufacturer buys portion of MAC address space (to assure uniqueness)
- MAC flat address is portable (can move LAN card from one LAN to another)
- IP hierarchical address is not portable (depends on the IP subnet to which node is attached)
- Error Detection and Correction
 - EDC = error detection and correction bits
 - D = data protected by error checking
 - error detection is not 100% reliable
 - larger EDC yields better detection and correction
- Single Bit Parity
 - even parity: add on a parity bit to make the parity even
 - this will not detect when two bits have been erroneously changed
- 2 dimensional parity
 - row parities - one bit even parity
 - column parities - one bit even parity
 - the parity bits also have a single parity bit (at the corner)
 - can detect and correct single bit error, can detect two/three errors, but not all four errors
 - first, row parity bits, then column parity bits, finally overall parity bits
- Checksum - should be 0
- Cyclic Redundancy Check
 - certain errors are observed frequently "in the wild"
 - CRCs of length 32 can detect every burst error of length 33 or less and burst errors greater than 33 with probability $1 - \frac{1}{2^{32}}$
 - CRCs cannot correct errors
 - parametrized by constants G and r , $r + 1$ is the length of G ($r = 8, 12, 16, 32$)
 - G is the generator
 - the sender wants to send D
 - the sender chooses r CRC bits, R , such that $\langle D, R \rangle$ is **exactly divisible** by G (modulo 2)

- want $D \cdot 2^r \oplus R = nG$

Nov. 7th - Module 7.2: Access Control and ARP

- Terminology
 - the networks that we talk about at the link layer are often called Local Area Networks (LANs)
 - historically, they were limited to a small area and a relatively small number of adapters
 - often use a broadcast medium for communication
- Access control
 - many links are half-duplex - multiple adapters can transmit, but only one at a time
 - some are full-duplex - both sides can transmit at the same time without interference
 - make two assumptions
 - can tell if someone else is sending by listening
 - can tell if your transmission overlapped some other transmission (a *collision* happened) again by listening
- Random Access Control
 - when you have something to send, just send it
 - if a collision is detected, just try again later after random delay
- CSMA - Carrier Sense Multiple Access
 - listen before sending, only send if no one else is
 - Add collision detection
 - while sending, listen for a collision
 - if so, abort transmission early
 - wastes less time than continuing to send the whole frame
 - matches human conventions for human conversation
- How long to wait before trying again?
 - binary exponential backoff
 - choose a random number in the range $2^n - 1$
 - where n is the number of times you have collided trying to send

this frame

- Turn-based Access Control
 - sender takes turns
 - if they have nothing to send right now, pass on their turn
 - Centralized control
 - a single node receives whose turn it is
 - either it polls everyone, or
 - there is a way for an adapter to signal that it wants a turn
 - used in Bluetooth, the central or master device controls who sends and when
 - Decentralized control
 - a "token" is passed between senders
 - only send when you have the token
- Switches (Layer 2 devices)
- Switched links
 - each wire has only 2 ends
 - if engineered right, can be full-duplex
 - no need for access control
 - the central switch implements the broadcast behaviour expected in a LAN
 - every incoming frame is duplicated to every wire (except the one it came on)
- How does it work?
 - 3 columns
 - each time a frame is received on an interface, the switch looks at both the source and destination MAC address
 - if the source address appears in the table, update the interface and the time
 - if the source address does not appear in the table, add an entry with the address, the interface, and the time
 - if destination address not in the table, send to every interface except the one it came from

- if interface for destination is in the table and equal to the interface it came from, discard the frame
- if interface for destination is in the table and different from where it came from, send the frame to the designated interface
- different MAC address, same interface number - maybe a switch that has connected multiple devices
- we never put the broadcast address **FF-FF-FF-FF-FF-FF** in the table
- delete entries in the table whose time is too long ago
- Switch vs Router
 - a switch is a link layer device
 - can send frames only between directly connected interfaces
 - supports broadcast
 - a router is a network layer device
 - can send datagram to any host in the Internet
 - does not support broadcast
- Address Resolution Protocol - ARP (Resolving IP address to MAC address)
 - A wants to send datagram to B, but does not know B's MAC address
 - A **broadcasts** ARP query packet, containing B's IP address
 - dest MAC address: **FF-FF-FF-FF-FF-FF**
 - all machines on LAN receive ARP query
 - B receives ARP query, replies to A with its MAC address
 - frame sent to A's MAC address
 - A caches B's IP and MAC address pair in its ARP table
 - ARP table is a soft state: information that goes away unless refreshed (each entry in the table has a time limit)
 - "plug-and-play": nodes create their ARP tables without intervention from network administrators
- Fun Fact about ARP
 - stateless, always read a response even if it did not make a request
 - not authenticated, anyone can ARP
 - can be spoofed - can attempt to "hijack" another host's IP address by responding to ARP requests, or sending replies that no one asked for
 - works in a single "broadcast domain"

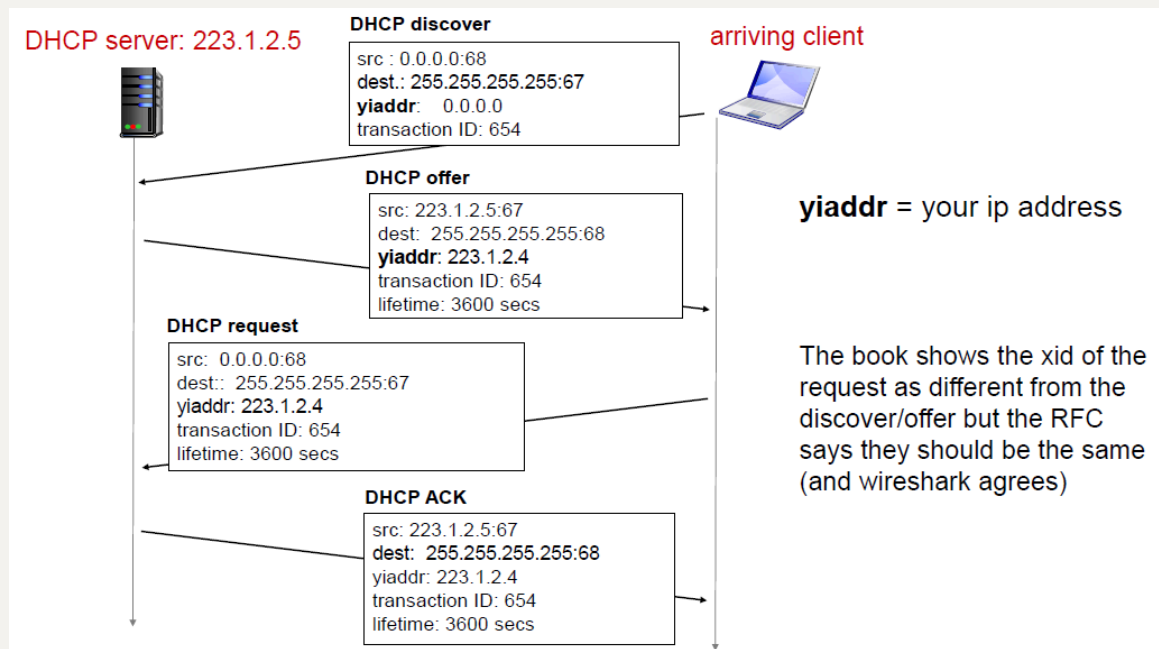
- reverse-ARP used to be used to get an IP address but is now obsolete, we use DHCP instead

Nov. 14th - Module 7.3: Dynamic Host Configuration Protocol

- DHCP
 - Goal: allow host to **dynamically** obtain its IP address (and other configuration information) when it joins a network
 - support for mobile users who want to join network
 - Built on a client-server model, on top of UDP ports 67 (server) and 68 (client)
 - Basic messages
 - host broadcasts **DHCP discover**
 - DHCP server responds with **DHCP offer**
 - host requests IP address with **DHCP request**
 - DHCP server sends **DHCP ack**
- Which of the following does a newly arriving device need when it joins a network?
 - An IP address, the subnet mask (need this to know if destination address is in the same address), the IP address of a router that can forward datagrams outside this network (need this to forward datagrams outside of the network)
 - No need for IP address of a DHCP server (**discover** broadcast was used)
 - Optional: the IP address of a DNS server (name resolving)
- Reverse ARP
 - the predecessor protocol to DHCP was called RARP
 - it only provided the newly arriving device an IP address
 - it was enough for many years because IPv4 was class-based, but we changed to classless inter-domain routing.
- DHCP steps
 - **discover**: client asks server for an IP address
 - **offer**: server provides a possible IP address to client
 - **request**: client accepts the offer and requests to be assigned the IP

address

- **ack**: server acknowledges request and assigns new IP address to client

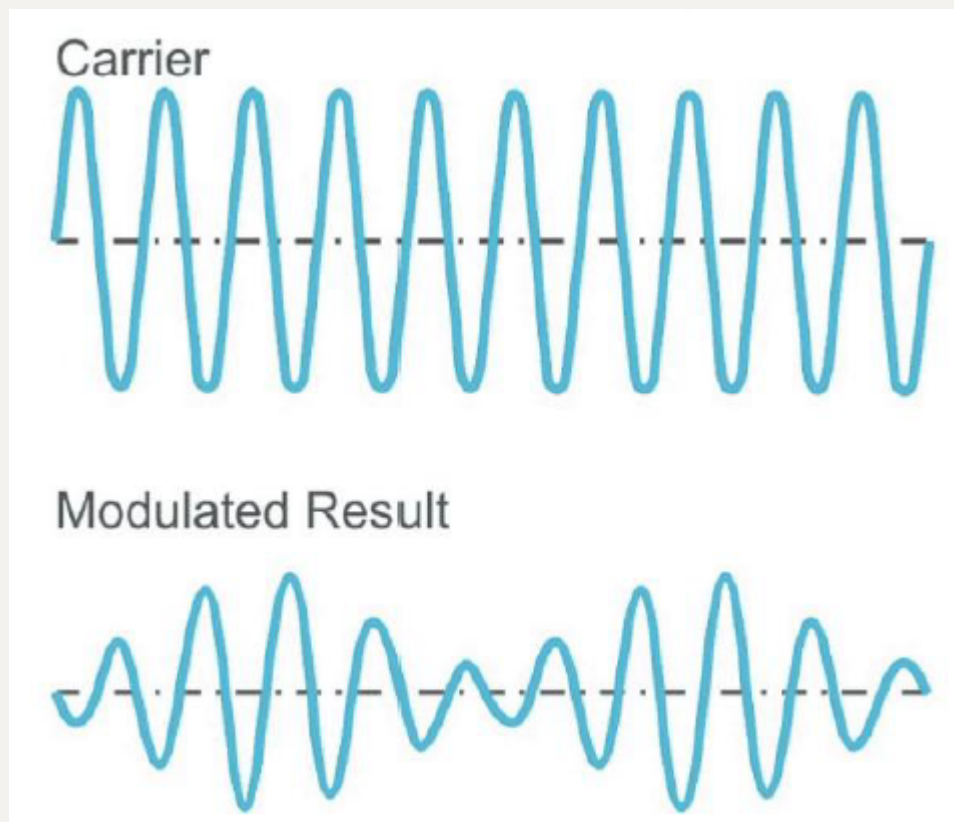


- Transaction ID is used for sender to know the IP address is offered to it
- DHCP offer/ack includes
 - unique IP address for client
 - netmask for local network
 - lease time
 - how long can you use the IP address before renewing (in seconds)
 - routing information
 - usually IP address for only one catch-all router (default gateway) is provided
 - Host name, domain name (optional)
 - Name (DNS) server (optional)
- DHCP questions
 - lease time? we can reuse IP address.
 - more than one DHCP server? client chooses which offer to accept (one DHCP server knows client declined their offer because of a different IP address request, but have same transaction ID)
- After DHCP process finishes, and an application has data to send, which of the following **still needs to be retrieved**?
 - The link-layer address of the router (MAC address of the router)
- When the lease runs out

- near expiration time, client just sends request again (when half the lease is left).
- if no response, repeat again when one eighth of the lease is left.
- When clients wants to leave
 - send **DHCP release** to notify it is done with lease

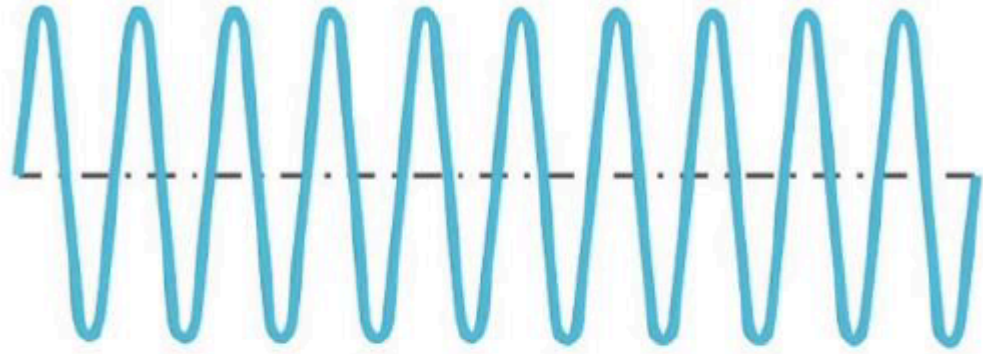
Nov. 17th - Module 7.4: Physical and Link Layer Issues

- The Physical Layer
 - many different media that can support networking
 - co-axial cable, twisted pair (phone cables), fiber, radio (Wi-Fi, Bluetooth, Cellular, Satellite)
- Modulation
 - how bits are put on the medium is an EE issue, but generally there is a carrier signal which is modulated to contain the data
 - AM - amplitude modulation
 - FM - frequency modulation
 - PM - phase modulation
- AM - amplitude modulation

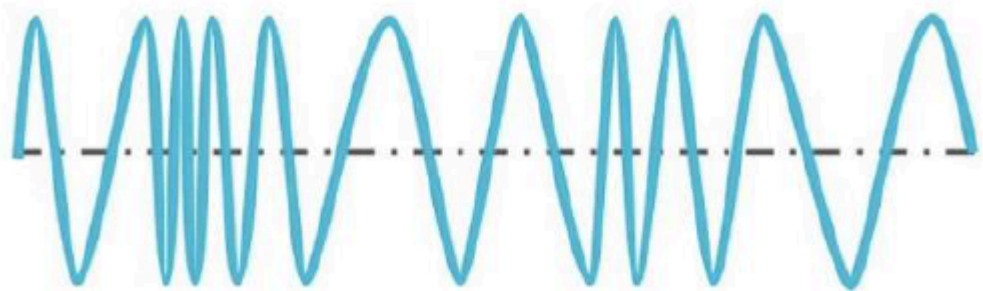


- FM - frequency modulation

Carrier

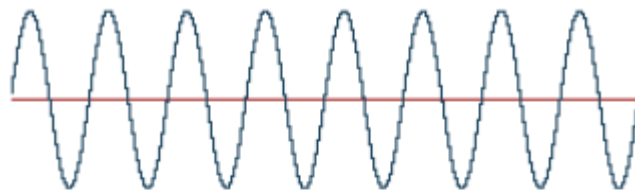


Modulated Result

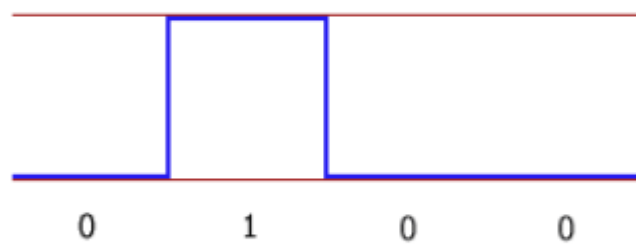


- PM - phase modulation

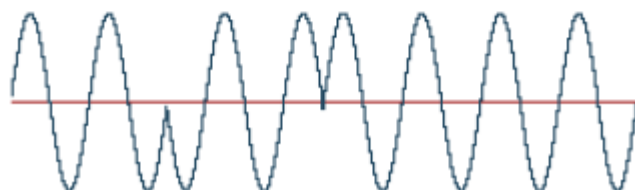
Carrier



Modulating Wave (digital)



Modulated Result



- Sending the bits
 - almost all physical networks send the bits of the message one at a time on the medium
 - code-division multiplexing complicates this because a single code symbol generally encodes more than one bit
 - the timing of when bits are sent is determined by the bandwidth of the medium
 - 10 Mbps: one bit every 100 ns, 100 Mbps: one bit every 10 ns
 - 1 Gbps: one bit every 1 ns, 10 Gbps: one bit every 100 ps
- A scenario
 - a sender is sending 8 bits to a receiver
 - transmission time is 1 time unit per bit
 - 100 Mbps $\rightarrow$ 1 bit every 10 ns = 80 ns
 - distance between sender and receiver is 10 m (propagation delay is 5 time units = 50 ns)
- A problem with LANs
 - having a single large LAN is sometimes less than ideal
 - all broadcast traffic goes to every adapter
 - privacy and security
 - anyone using Wireshark can see any frames
 - creating separate LANs would solve the problem
 - one LAN for each group in a company, for example
 - but each LAN needs a switch
 - many small switches are more expensive than a single large switch
- What if we could...?
 - have multiple virtual LANs using a single switch
 - enter the Virtual Local Area Network - VLAN
 - since switches contain software forwarding already, it is possible to create multiple VLANs on a single switch
 - identified by port
 - identified by MAC address
 - with a bit more software, the switch hardware can also be the router

that connects these VLANs together

- Data center networking
 - 10,000 - 100,000 of computers in a small space
 - distributed algorithms run on these computers
 - spark, hadoop, tensorflow
 - communication is a key component
 - how are these things connected together?
- Data Center Characteristics
 - network speeds up to 100 Gbps
 - 20-40 servers per rack
 - Suppose all 40 servers in a rack need to talk to a different server **in the same rack**. What is the bandwidth of each connection? (Assume 100 Gbps bandwidth)
 - 100 Gbps
 - if on **different racks**: 2.5 Gbps

Nov. 19th - Module 8.1: Security - Introduction

- What security issues have we seen?
 - SMTP spoofing, False BGP routing advertisements, DNS hijacking, No privacy in Ethernet etc, IP address spoofing
- Principles of security
 - Confidentiality: only the sender and the intended receiver should "understand" message contents
 - sender *encrypts* message
 - receiver *decrypts* message
 - Authentication: the sender and receiver want to confirm each other's identity
 - Message integrity: the sender and receiver want to ensure that the message is not altered (in transit, or afterwards) without detection
 - Access and availability: services must be accessible and available to users
- Privacy and Security
 - TCP provides reliable, in-order delivery data

- it does not provide security or privacy
- What level should be responsible for security?
 - A new network-layer protocol (IPsec)
 - A new transport-layer protocol (QUIC)
 - A new "pseudo-layer" between transport and application (SSL, TLS)
 - responsibility of the application (SSH)
- The Actors
 - Most network security literature uses the names:
 - Alice, Bob want to communicate "securely"
 - Trudy is an adversary who "attacks" the communication between Alice and Bob
 - What can Trudy do?
 - Eavesdrop: sniff and record messages on the channel
 - Actively insert messages in communication
 - Impersonation: can fake (spoof) an entity
 - Hijacking: "take over" ongoing connection by removing sender or receiver, inserting themselves in place
 - Denial of service: prevent service from being used by others (e.g. by overloading resources, dropping packets)
- Confidentiality
 - no one but the sender and the receiver should be able to interpret the content of the message
 - thought experiment: where in "real life" do we have or expect to have confidentiality?
- Encryption
 - an encryption algorithm comprises: a method for encrypting the data, a method for decrypting the data, a secret key used in the decryption/encryption method
 - Caesar cipher
 - encryption: replace all letters by the corresponding letter K positions ahead in the alphabet
 - decryption: repeat process with $(-K)$
 - secret key is K

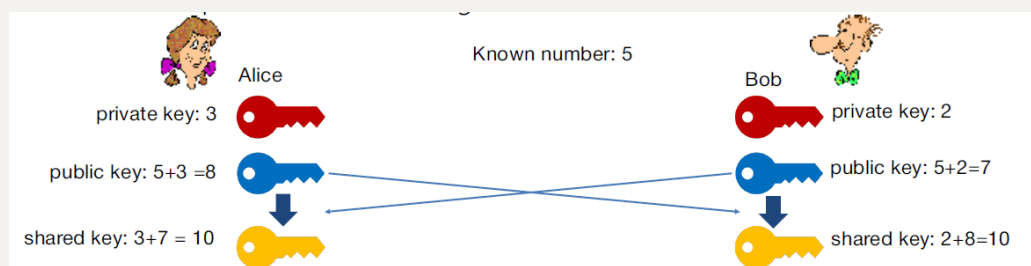
- $RotK$ is the Caesar cipher with a key of K
- e.g. $TURNIP \rightarrow Rot13 \rightarrow GHEAVC$
- Encryption approach
 - naive approach: security by obscurity
 - assume attacker is unaware of the decryption method
 - unsafe: good encryption algorithms are eventually made public
 - attackers can "pick a lock"
 - better approach: secret key
 - assume attacker knows the method, but not the key
 - decryption is impossible without the key
- Language of cryptography
 - m : plaintext message
 - $K_A(m) = c$: ciphertext c , encrypted with key K_A
 - $K_B(c) = K_B(K_A(m)) = m$
- Breaking an encryption scheme
 - Option 1: brute force, search through all keys
 - Option 2: statistical analysis (look for patterns)
 - A number of classes of attack based on how much Trudy knows
 - Ciphertext-only attack: Trudy has ciphertext, but not plaintext (knows $K_A(m)$, but not m)
 - Known-plaintext attack: Trudy has some ciphertext with its plaintext (for some m it knows $K_A(m)$), wants to break other ciphertexts
 - Chosen-plaintext attack: Trudy has the ability to encrypt any plaintext (knows K_A , or can trick Alice into encrypting any message), but does not have key for decryption

Nov. 21st - Module 8.2: Encryption

- Symmetric key cryptography
 - Bob and Alice share same (symmetric) key: K_S
 - key is knowing substitution pattern in mono-alphabetic substitution cipher

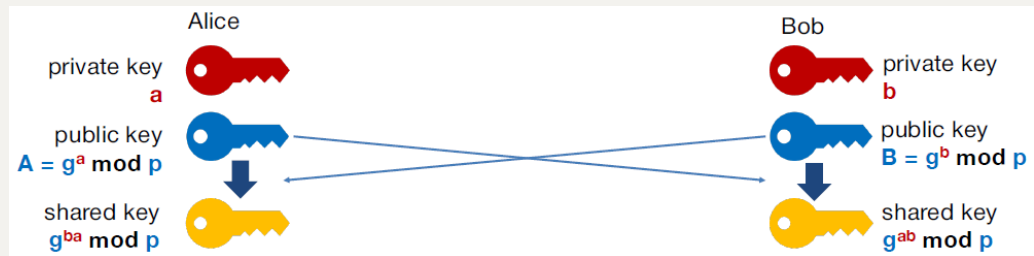
- method may be different (opposite) for decryption, but uses same key
- Simple Encryption Scheme
 - Substitution Cipher
 - substitute one thing for another
 - "thing" can be a byte, block, word, etc
 - monoalphabetic cipher: substitute one letter for another
 - encryption key: mapping from one set to another
 - similar to Caesar cipher, but the mapping is less regular
 - Slightly better encryption
 - several substitution ciphers
 - predictable pattern of ciphers (cycling pattern, algorithm that decides next pattern)
 - for each new symbol, use next substitution pattern
 - encryption key: all ciphers, plus pattern
- Block Ciphers
 - message is broken into blocks
 - each block is encrypted and decrypted separately
 - encryption method can be as simple as substitution cipher (substitution table for 64-bit blocks would require 2^{64} entries)
- DES: data encryption standard
 - 56-bit symmetric key, 64-bit plaintext input blocks
 - block cipher: substitution derived from symmetric key
 - cipher block chaining: initial vector derived from symmetric key
 - not considered secure any longer
 - can be decrypted with brute force within a day since 1999
 - 3DES: more secure
 - encrypt 3 times with 3 different keys
- AES: advanced encryption standard
 - symmetric key, replaced DES as NIST standard in 2001
 - 128-bit block cipher
 - 128-, 192-, 256-bit key
 - difference is just the number of rounds of translation

- way more secure than DES
 - brute force decryption that takes 1 second for DES would take 149 trillion years for 128-bit AES
- Block ciphers plus
 - keeping the same substitution can be risky
 - allows for statistical analysis of common substitutions
 - to avoid this we can change the substitution for every block
- Cipher-Block Chaining
 - first block is XORed with an arbitrary (randomly chosen) number known by both parties (initialization vector or IV) and then encrypted using a substitution cipher with K_S
 - following blocks are XORed with previous block, then encrypted
 - $C[0]: \text{IV}$
 - $C[1] = K_S(M[0] \oplus C[0])$
 - $C[n+1] = K_S(M[n] \oplus C[n])$
 - decryption: apply the (reverse) substitution using K_S then XOR with previous block
- Who knows the key?
 - both receiver and sender must know the key
 - sender needs it for encryption, receiver needs it for decryption
 - each connection pair must have its own key
 - sharing keys with other peers dilutes the trust
 - what if sender and receiver never negotiated a key before?
 - key can be generated when a connection first starts



- Can work if we won't be able to know private key seeing public key
- Need to easily get the public key from the private key but cannot get the private key from the public key alone
- $(g^a)^b = (g^b)^a$

- Diffie-Hellman Key Exchange
 - exponentiation modulo algorithm
 - Alice and Bob agree on key generators: p (prime number) and g (exponentiation base)



Nov. 24th - Module 8.3: Asymmetric Encryption

- Asymmetric cryptography
 - some algorithms use a pair of keys (e.g. RSA)
 - if one key is used for encryption, the other is used for decryption

$$K_1(K_2(m)) = K_2(K_1(m)) = m$$
 - you can generate a pair of keys, but one key cannot be obtained from the other in reasonable computation time
 - one key is kept private, one key is made public
 - usually denoted K^- and K^+
 - if Alice encrypts a message with Bob's public key, Bob can decrypt the message
 - if Alice encrypts a message with Alice's private key, everybody else can decrypt it
 - if sender encrypts with public key, only private key can decrypt it
 - used for confidentiality
 - $K^-(K^+(m)) = m$
 - alternatively, $Dec(K^-, Enc(K^+, m)) = m$
 - if owner encrypts with private key, public key can decrypt it
 - used for authentication, so call sign/verify instead of encrypt/decrypt
 - $K^+(K^-(m)) = m$
 - alternatively, $Dec(K^+, Enc(K^-, m)) = m$
- Algorithm

- at least require $K_B^+(x)$ and $K_B^-(x)$ such that $K_B^-(K_B^+(m)) = m$
- opposite is ideal for authentication, but not required for encryption
- given public key, should be computationally unfeasible to compute the private key
- RSA
 - Creating key pair
 - choose two large prime numbers p, q (e.g. 1024-bits each)
 - compute $n = pq$, $z = (p - 1)(q - 1)$
 - choose $1 < e < n$, such that $\gcd(e, z) = 1$
 - choose $d < z$ such that $ed \bmod z = 1$
 - public key is $K_B^+ = (n, e)$, private key is $K_B^- = (n, d)$
 - Encryption, message m
 - $c = m^e \bmod n$
 - Decryption, ciphertext c
 - $m = c^d \bmod n$
 - security depends on factoring n is computationally hard
 - bonus feature: $K_B^-(K_B^+(m)) = K_B^+(K_B^-(m)) = m$
- Message integrity: signatures
 - simple digital signature for message m
 - Bob signs m by encrypting with his private key: $K_B^-(m)$
 - Bob sends both the original message m and the signature
 - Alice can check if $m = K_B^+(K_B^-(m))$
- Message digest
 - computing the signature by applying a private key to a long message is computationally expensive
 - alternative: compute a fixed-length "fingerprint"
 - apply hash function H to message m , giving a fixed size message digest $H(m)$
- Signed message digest
 - sign only the hash result
 - Bob sends message m and signed digest $K_B^-(H(m))$
 - Alice receives m and computes $H_{new}(m)$

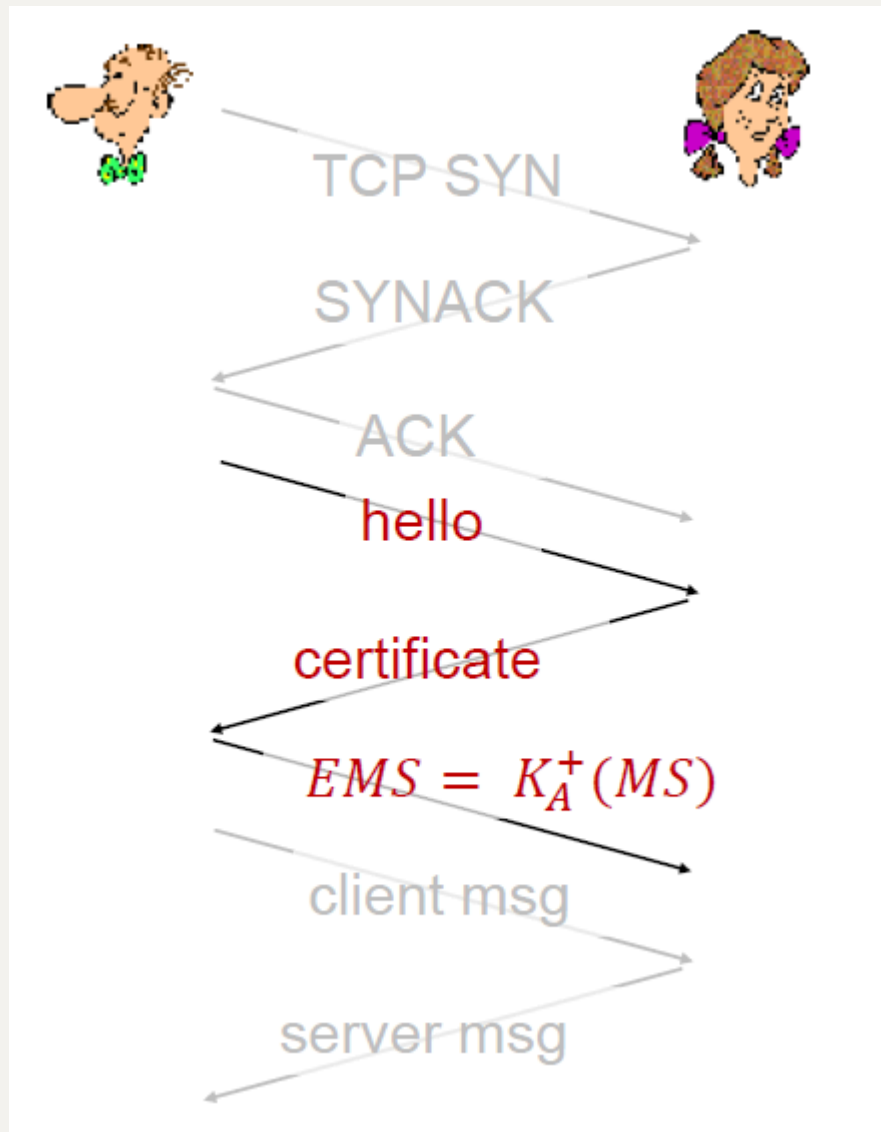
- Alice receives $K_B^-(H(m))$ and computes $K_B^+(K_B^-(H(m)))$
- check if the Alice computed values match
- Secure Hash
 - given x , it is computationally infeasible to find m such that $H(m) = x$
 - Given m , it is computationally infeasible to find $m' \neq m$ such that $H(m) = H(m')$
 - simple checksums as hash
 - it produces fixed-length digest of message, but still easy to find another message with same hash value
- Better Hash Function Algorithms
 - MD5 hash function widely used
 - compute a 128-bit message digest in 4-step process
 - hard to compute message from hash
 - SHA-1 is more secure
 - US-NIST standard
 - 160-bit digest
 - newer alternatives: SHA-256, SHA-512
- Message Authentication Code (MAC)
 - alternative to signed message digest
 - shared secret s is used between parties (similar to symmetric cryptography)
 - hash is computed not on message m , but on $m + s$
 - Bob sends message m and $h = H(m + s)$
 - Alice receives (m, h) and computes $H(m + s)$
 - if $h = H(m + s)$, message is considered signed
 - faster, since no encryption is necessary
 - HMAC: popular MAC standard
- Confidentiality, Authentication and Message Integrity
 - sometimes integrity without confidentiality is acceptable
 - confidentiality without integrity is not useful in practice (CBC vulnerability)
 - authenticated encryption (AE)

- MAC-then-encrypt (SSL)
- Encrypt-then-MAC (IPSec) (Only decrypt when authentication passed)
- Encrypt-and-MAC (SSH)
- Certification Authority (CA)
 - entity (person, website, etc) registers public key with CA, provides some "proof of identity"
 - this process is usually performed offline
 - Certification Authority provides a certificate
 - binds public key to particular entity
 - signed by CA's private key
 - CA's public key is known ("trusted") to public
 - checking a certificate
 - when Alice wants Bob's public key
 - Bob provides a certificate
 - certificate is signed by CA
 - Alice applies CA's public key to confirm
 - certificate contains Bob's public key
- Problems
 - computationally expensive
 - only use it to establish secure connection and for authentication

Nov. 26th - Module 8.4: Authentication and TLS

- End-point authentication
 - Bob wants Alice to prove her identity
 - Option 1: Alice sends a message with identification
 - Option 2: Alice includes own IP, known to Bob
 - Option 3: use a password
 - Option 3.1: use an encrypted password (Trudy can just need to send the encrypted password, no need to know original password)
 - all the options above are prone to easy spoofing

- Option 4: Bob sends one-time message R to Alice ("nonce"), Alice returns same message encrypted with shared key
 - Bob can confirm Alice knows the shared encryption key
 - Requires a pre-established secret key (key exchange through Diffie-Hellman)
 - Password could be used, but relies on the password itself being secure enough
- Option 5: Bob sends one-time message R to Alice, Alice returns same message encrypted with K_A^-
 - Bob then requests Alice's public key
 - only reliable if Bob has a way to confirm K_A^+ is actually Alice's
 - subject to man-in-the-middle attack
- Certification Authority - checking a certificate
 - when Bob wants Alice's public key, Alice provides a certificate signed by CA, Bob applies CA's public key to confirm certificate's authenticity
 - certification contains Alice's public key
- Building a Security Protocol
 - Handshake: Alice, Bob use their certificates to authenticate each other, and private keys to share secret
 - Key derivation: Alice and Bob use shared secret to derive keys
 - Data transfer: series of messages
 - Connection termination: securely close connection
- Handshake



- Bob establishes TCP connection with Alice
- Bob verifies Alice's identity (certificate)
- Bob sends Alice master secret key MS
 - used to generate all other keys for session
- Potential issue
 - 3 RTTs before client can start receiving data
- Key Derivation
 - K_B : encryption key for data sent from Bob to Alice
 - M_B : MAC key for data sent from Bob to Alice
 - K_A : encryption key for data sent from Alice to Bob
 - M_A : MAC key for data sent from Alice to Bob
 - Keys derived from predetermined key derivation function (KDF)
 - process could be as simple as splitting MS into 4 parts

- Data Transfer
 - TCP provides byte stream abstraction
 - data is broken into individual "records"
 - each record carries a MAC, created using M_A or M_B
 - receiver can act on each record as it arrives
 - to send message m with length l from Alice to Bob:

$$K_A(l, m, H(m + M_A))$$
 - but we should encrypt, then MAC, so send $H(K_A(l, m) + M_A), K_A(l, m)$
 - possible attack on data stream
 - reordering (attacker intercepts TCP segments and manipulates sequence numbers)
 - replay (same TCP message sent twice, manipulating TCP)
 - solution: include sequence number n in MAC computation
 - use $H(K_A(l, m) + M_A + n)$
- Connection Termination
 - possible attack on data stream
 - attacker forges TCP connection close segment (truncation attack)
 - solution: special message type for closure
 - to send regular message m with length l and sequence number n from Alice to Bob

$$H(K_A(l, 0, m) + M_A + n), K_A(l, 0, m)$$
 - to send closing message

$$H(K_A(l, 1, m) + M_A + n), K_A(l, 1, m)$$
- Transport-Layer Security (TLS)
 - above transport layer, technically application layer, in-between
 - replaces Security Socket Layer (SSL) protocol
 - built to support any type of application
 - confidentiality: symmetric encryption
 - integrity: MAC
 - authentication: via public key cryptography and certificates
 - supports several algorithm for: key generation, encryption, MAC, digital

- signature
- cipher suite: choice of algorithms negotiated during handshake
 - client sends supported cipher suites
 - server chooses one
- QUIC
 - transport layer protocol running on UDP
 - provides reliability of TCP, plus security of TLS

Nov. 28th - Module 8.5: IPSec, VPN, Firewall, IDS

- Network Layer Security: IPSec
 - provides datagram-level encryption, authentication, integrity
 - for both user traffic and control traffic (e.g. BGP, DNS, ICMP)
 - can be used to implement a VPN (there are alternatives using protocols like TLS)
 - summary
 - peers can be end systems, routers, firewalls, or a mix of these
 - association can be done manually or using protocols like IKE (Internet Key Exchange)
 - policies, algorithms, secret keys, identification
 - provides source authentication, integrity and confidentiality
- Virtual Private Network (VPN)
 - Motivation
 - company with multiple locations wants everything to appear as one big network
 - workers want access to resources restricted to company's internal network
 - students want access to restricted material inside UBC network
 - users want to bypass regional blocks
 - Solution: pretend you are somewhere else
 - virtual network interface cards on two end point systems
 - Encapsulation
 - virtual end points establish a software association between

- them (called a tunnel)
 - routing rules send traffic to virtual card
 - virtual card encapsulates IP message and sends it into the virtual connection
 - receiver receives this IP message and sends it through its own network
- the packet is transferred through the VPN gateway because it has a lower metric
- Privacy
 - the payload that flows through the VPN tunnel can be encrypted so no one can see the contents
- Security in other protocols
 - DNS → DNSSEC
 - adds authentication for DNS servers
 - uses PKI (public-key infrastructure aka asymmetric cryptosystem)
 - each DNS server signs its RRs
 - no confidentiality
 - BGP → BGPsec
 - adds integrity for BGP paths, uses PKI, router adds AS number of the AS it's sending the update to and signs its update, no confidentiality
 - E-mail → PGP
 - confidentiality, integrity, and authentication
 - every user has a unique public-private key pair
 - one symmetric key for every email
 - sender encrypts message with session key and the session key with the recipient's public key
- Access and Availability: Firewalls
 - operational security (manage availability and access)
 - firewall isolates internal network from public Internet
 - all traffic from outside to inside passes through the firewall
 - allows some datagrams to pass, blocks others
 - typically located in a router

- the accessibility is based on set of rules (access control lists)
 - prevent illegal access/modification of internal data
 - allow only authorized access to inside network
 - prevent DoS attacks
- Stateless Packet Filtering
 - filter packet by packet, decision to forward or drop is based on the following
 - source/destination IP address and protocol
 - source/destination TCP/UDP port numbers
 - ICMP message type
 - TCP flags, and other header values
 - can be used to
 - block inbound TCP segments with $ACK = 0$
 - block inbound or outbound TCP segments with port = 23 (Telnet)
 - block inbound ICMP messages to broadcast IP address
 - block outbound ICMP expired TTL messages (traceroute uses such ICMP messages)
- Access Control Lists
 - table of rules, applied in order to incoming packets
- Stateful Packet Filtering
 - track status of every TCP connection
 - track SYN and FIN messages, incoming and outgoing
 - block packets that don't "make sense"
 - track timeouts
 - track status of some UDP connections
 - response should only come if a request comes from inside
 - ACL augmented to indicate need to check connection state table
- Application Gateways
 - filter packets based on application data
 - e.g. restriction based on users rather than IP addresses, restriction based on DNS hosts or URL patterns

- can be implemented with a proxy-like structure
- Intrusion Detection Systems
 - packet filtering can operate on headers only
 - no correlation check among sessions
 - deep packet inspection: look at packet contents
 - check packets against known attack strings ("signature based")
 - identify statistically unusual traffic ("anomaly based")
 - examine correlation among packets
 - identify port scanning, networking mapping, DoS attacks

Dec. 1st - Module 8.6: Availability

- How can Trudy make a service unavailable?
 - crash a service or the host it runs on (depends on some vulnerability in the service or the host)
 - overwhelm the service with data so it does not have time or other resources to do its "normal" work
 - denial of service attack can target any resource (bandwidth, connections, memory)
- SYN flood attack
 - Trudy sends MANY SYN messages to "initiate" connections, and occupy the resources to prevent a normal user from connecting with the server
 - limiting the number of SYN from one user (might work, Trudy can work around, say, by spoofing)
 - not allocating memory before confirmation (works, but tricky in the sense that most connections are legitimate)
- SMURF attack
 - Trudy sends ICMP ECHO REQUEST with spoofed SRC IP, and Alice gets a huge flow of unwanted traffic
- DNS amplification attack
 - Trudy sends DNS REQUEST with spoofed SRC IP that requests for all records of a server, and floods Alice's traffic
- DOS mitigations

- disable broadcast addresses
- firewalls: reject UDP traffic from outside, rate limit ICMP pings
- source IP verification: ISPs reject traffic with spoofed IP addresses
- reduce open DNS resolvers: respond only to queries from trusted sources
- Amplification Effect
 - most effect denial-of-service attacks exploit amplification
 - an attacker triggers their botnet to all send data to a particular service or host (1 attacker → 10000 bots)
- Malware
 - anything designed to compromise or harm a host
 - Goals: accessing private data, destroying data, holding data hostage, controlling the host for other purposes
- Virus
 - malware that depends on a user action to infect the host
 - compromised: executable files, documents, email messages
 - phishing emails containing links
- Worm
 - malware that infects a host without user action
 - takes advantage of software with vulnerabilities
 - especially vulnerable is software that accepts data across the network: Web servers, SSH servers, Database servers
- Botnet
 - a collection of compromised hosts used for evil purposes
 - sending spam email
 - organized attacks on other services
- DDoS Mitigations
 - preventions: change default passwords on IoT devices, update OS, do not click on suspicious links
 - mitigations take time: monitor traffic and detect attack, activate firewalls/rate limits, find root of botnet

Dec. 3rd - Module 9: Wrapup

- Module 1: Design
 - The Internet is a network of networks
 - Design goals: reliability, flexibility, decentralized management, cost effectiveness, accountability
 - protocols, layers, circuit switching & packet switching
- Module 2: Performance
 - Latency, RTT, Bandwidth, Throughput, Goodput, Jitter, Queuing delay formula
- Module 3: Application layer
 - P2P vs client-server
 - Sockets
 - HTTP, DNS, E-mail (IMAP, POP3, SMTP)
 - Bittorrent and Bitcoin
- Module 4: Transport layer
 - Ports
 - UDP
 - Alternating Bit Protocol
 - Sliding windows: GBN and SR
 - Flow and congestion control
 - TCP
- Module 5: Network Layer
 - Autonomous Systems
 - IP addresses: classes and CIDR
 - Forwarding
 - Routing: Intra-domain and Inter-domain
- Module 6: NAT - Network Address Translation
- Module 7: Link layer
 - MAC addresses
 - Link layer frame format
 - Error detection: parity & CRC

- Switches and Forwarding
- ARP, DHCP
- VLANs, & Data center networking
- Module 8: Security
 - Confidentiality, Authentication, Integrity, Availability
 - Shared & Public/Private key cryptography
 - Authentication & Message Integrity
 - Message Authentication Codes
 - Certificates
 - TLS & IPsec
 - Denial of Service attacks